



TRABALLO FIN DE GRAO
GRAO EN ENXEÑARÍA INFORMÁTICA
MENCIÓN EN SISTEMAS DE INFORMACIÓN



Modernización del módulo Report Service en la plataforma BIC para el cliente AESLA

Estudiante: Andrea Liñares Castro

Dirección: Fernando Silva Coira
Ángel Fernández Formoso

A Coruña, septiembre de 2026.

A GBTEC Software AG, por la oportunidad de crecer y aprender en un entorno profesional.

Agradecimientos

Quiero agradecer a GBTEC Software AG, y especialmente a Ángel, mi tutor, y a Ana, mi compañera, la oportunidad de realizar este proyecto en un entorno profesional, así como la confianza, la orientación y el apoyo recibidos durante estos meses.

También agradezco a mi familia, a mis amigos y a mi pareja su apoyo constante, su paciencia y sus palabras de ánimo durante la elaboración de este Trabajo de Fin de Grado. Gracias igualmente a mis compañeros de clase por todos estos años de aprendizaje compartido, por la ayuda mutua y por los buenos momentos vividos durante la carrera.

Resumen

En el contexto de la gestión de procesos empresariales y la generación automática de informes ejecutivos, este Trabajo de Fin de Grado presenta la modernización del módulo Report Service para AESLA, desarrollado por GBTEC Software AG. El proyecto aborda la optimización del flujo de generación de informes en formato Excel mediante la incorporación de almacenamiento histórico, procesamiento asíncrono, control de duplicidad y mejoras en la experiencia de usuario. Para ello, se aplicó una metodología ágil iterativa e incremental, utilizando tecnologías como Spring Batch, Flyway, PostgreSQL, Elasticsearch y Angular. Las mejoras contribuyen a aumentar la mantenibilidad, trazabilidad y calidad del servicio, al tiempo que amplían sus capacidades funcionales y mejoran la gestión de los informes generados. Como resultado, el Report Service evoluciona desde un sistema centrado en la generación y descarga inmediata hacia una solución más completa, que incorpora almacenamiento histórico, seguimiento del proceso y recuperación posterior de los informes.

Abstract

Within the context of business process management and automated executive report generation, this Bachelor's Thesis presents the modernization of the Report Service module for AESLA, developed by GBTEC Software AG. The project focuses on improving the Excel report generation workflow by introducing historical storage, asynchronous processing, duplicate-request control, and enhancements to the user experience. An iterative and incremental agile methodology was followed, using technologies such as Spring Batch, Flyway, PostgreSQL, Elasticsearch, and Angular. These improvements contribute to increasing the maintainability, traceability, and quality of the service while extending its functional capabilities and improving the management of generated reports. As a result, the Report Service evolves from a system focused mainly on immediate report generation and download into a more complete solution that supports historical storage, generation tracking, and subsequent report retrieval.

Palabras clave:

- Generación automática de informes
- Procesamiento asíncrono
- Metodología ágil
- Spring Batch
- PostgreSQL
- Elasticsearch
- Angular
- Control de duplicidad
- Persistencia de informes

Keywords:

- Automatic report generation
- Asynchronous processing
- Agile methodology
- Spring Batch
- PostgreSQL
- Elasticsearch
- Angular
- Duplication control
- Report persistence

Índice general

1	Introducción	1
1.1	Motivación	1
1.2	Objetivos	6
1.3	Aportación personal al proyecto	7
2	Fundamentos tecnológicos	8
2.1	Estado del arte	8
2.2	Tecnologías utilizadas	11
2.2.1	Servidor web	11
2.2.2	Cliente web	11
2.2.3	Base de datos	12
2.2.4	Infraestructura y Despliegue	12
2.2.5	Herramientas de apoyo	12
3	Metodología y planificación	14
3.1	Metodología de desarrollo	14
3.1.1	Adaptación al proyecto	15
3.2	Planificación y seguimiento	15
3.2.1	Planificación inicial	16
3.2.2	Seguimiento real	16
4	Análisis	19
4.1	Situación inicial y alcance	19
4.2	Actores y sistemas involucrados	20
4.3	Requisitos	20
4.3.1	Requisitos funcionales	20
4.3.2	Requisitos no funcionales	21
4.4	Historias de usuario	22

4.5	Casos de uso	23
4.6	Interfaz original y limitaciones	23
4.7	Trazabilidad entre limitaciones y requisitos	24
5	Diseño de la solución	26
5.1	Arquitectura tecnológica del sistema	26
5.1.1	Visión general de la arquitectura	26
5.1.2	Comunicación entre componentes	29
5.1.3	Justificación de la arquitectura elegida	29
5.2	Diseño de la aplicación	30
5.2.1	Diseño del cliente	30
5.2.2	Diseño del servidor	31
5.2.3	Diseño del procesamiento asíncrono	32
5.2.4	Diseño de la persistencia	33
5.3	Interacción cliente-servidor	34
5.3.1	Inicio de una generación	35
5.3.2	Seguimiento de la generación	36
5.3.3	Consulta del historial	37
5.3.4	Descarga de un informe	39
5.3.5	Eliminación de un informe	40
5.3.6	Manejo de errores y respuestas HTTP	41
6	Implementación y validación	42
6.1	Organización de la implementación	42
6.1.1	Organización de los work items	42
6.2	Implementación del backend	43
6.2.1	Estructura física del proyecto	43
6.3	Unificación de la gestión de informes	43
6.3.1	Unificación del contrato de entrada	44
6.3.2	Normalización de parámetros	44
6.3.3	Separación entre flujo común y lógica específica	44
6.4	Implementación de la persistencia y el historial	45
6.4.1	Integración de Flyway	45
6.4.2	Migraciones implementadas	46
6.4.3	Consulta y recuperación de informes	48
6.5	Implementación de la generación asíncrona	48
6.5.1	Configuración de Spring Batch	48
6.5.2	Control de duplicidad mediante metadatos de Spring Batch	48

6.5.3	Comprobación de disponibilidad de datos	49
6.5.4	Generación y almacenamiento	51
6.6	Implementación del frontend	51
6.6.1	Estructura de módulos y componentes	51
6.6.2	Generación y seguimiento mediante polling	52
6.6.3	Página de historial	52
6.7	Mejoras de calidad y mantenimiento	53
6.7.1	Gestión de vulnerabilidades	53
6.7.2	Refactorización y reducción de duplicación	53
6.7.3	Observabilidad y soporte	54
6.7.4	Evolución de métricas de calidad	54
6.8	Incidencias y trabajo no realizado	54
6.9	Validación y pruebas	55
6.9.1	Relación entre requisitos y pruebas	55
6.9.2	Catálogo representativo de pruebas	56
6.9.3	Pruebas manuales y entorno de QA	56
7	Solución desarrollada	58
7.1	Resultado funcional global	58
7.2	Página de Generar	59
7.2.1	Funcionalidad	59
7.2.2	Comportamiento de la interfaz	60
7.3	Página de Historial	61
7.3.1	Funcionalidad	61
7.3.2	Comportamiento de la interfaz	62
7.4	Flujo funcional final	62
7.5	Preparación de la release	64
8	Conclusión y trabajo futuro	66
8.1	Evaluación del cumplimiento de objetivos	66
8.2	Conclusiones	67
8.3	Limitaciones	68
8.4	Posibles ampliaciones futuras	69
8.4.1	Actualización de la versión de Angular	69
8.4.2	Mejoras en la gestión del historial	69
8.4.3	Refactorización adicional del código backend	71
8.4.4	Funcionalidades avanzadas	71

A Evidencias técnicas	74
A.1 Arquitectura detallada	74
A.2 Evidencias de calidad de código	76
Lista de acrónimos	79
Glosario	80
Bibliografía	83

Índice de figuras

1.1	Mapa de presencia internacional de AESLA (Food Service Project). Fuente: elaboración propia a partir de [1].	1
1.2	Visión general de la BIC BPM Platform y de los módulos principales con los que interactúa el usuario. Fuente: elaboración propia.	3
1.3	Ejemplo de un proceso BPMN sencillo modelado en BIC Process Design. Fuente: elaboración propia a partir de [2].	4
1.4	Informe de tipo KPI con categoría de proceso Alternativa (todas las franquicias, sin rango de fechas). Fuente: resultado de la ejecución del Report Service.	5
1.5	Informe de tipo Producto con categoría de proceso Innovación (todas las franquicias, sin rango de fechas). Fuente: resultado de la ejecución del Report Service.	5
3.1	Diagrama de Gantt de la planificación inicial del proyecto.	16
3.2	Diagrama de Gantt correspondiente al seguimiento real del proyecto.	17
4.1	Diagrama de casos de uso principales del Report Service. Fuente: elaboración propia.	23
4.2	Interfaz original del Report Service antes de su modernización. Fuente: captura propia.	24
5.1	Arquitectura simplificada del flujo principal de generación de informes.	28
5.2	Diagrama de secuencia del proceso de generación y seguimiento de un informe, elaborado con Mermaid. Fuente: elaboración propia.	33
5.3	Diagrama de secuencia de la consulta y redescarga de informes desde el historial, elaborado con Mermaid. Fuente: elaboración propia.	40
7.1	Interfaz principal del módulo de generación de informes. Fuente: captura propia.	60
7.2	Vista de la página de historial de informes generados. Fuente: captura propia.	62

7.3	Flujo de interacción entre cliente y backend durante la generación, seguimiento y descarga de informes, generado con Mermaid. Fuente: elaboración propia.	63
8.1	Mockup de la interfaz de historial con funcionalidad de eliminación de informes. Fuente: elaboración propia.	70
8.2	Diálogo de confirmación para la eliminación de informes. Fuente: elaboración propia.	70
A.1	Arquitectura detallada del Report Service. Fuente: elaboración propia.	75
A.2	Captura de SonarCloud correspondiente a la cobertura inicial del proyecto: 17.7%.	76
A.3	Captura de SonarCloud correspondiente a la cobertura final del proyecto: 25.7%.	77
A.4	Captura de SonarCloud correspondiente a la duplicación inicial del proyecto: 15.8%.	77
A.5	Captura de SonarCloud correspondiente a la duplicación final del proyecto: 10.0%.	78

Índice de tablas

1.1	Principales aportaciones realizadas durante el proyecto.	7
2.1	Comparación cualitativa de soluciones relacionadas con el análisis y la generación de informes.	10
3.1	Comparación entre la planificación inicial y el seguimiento real del proyecto. .	18
4.1	Relación entre las limitaciones iniciales y los requisitos derivados.	25
5.1	Componentes principales y responsabilidades del Report Service.	27
5.2	Atributos de la entidad <i>Report</i> en <i>aesla_reports.report</i>	34
5.3	Operaciones REST principales utilizadas por el Report Service.	35
6.1	Migraciones de Flyway incorporadas durante el proyecto.	46
6.2	Evolución de las principales métricas de calidad medidas con SonarCloud. . .	54
6.3	Trazabilidad entre requisitos funcionales y pruebas realizadas.	55
6.4	Casos de prueba representativos del Report Service.	56
7.1	Comparación entre la versión inicial y la solución desarrollada.	59
7.2	Principales estados gestionados por la interfaz durante la generación.	61
8.1	Evaluación del cumplimiento de los objetivos del proyecto.	66

Capítulo 1

Introducción

1.1 Motivación

AESLA, S.A.B. de C.V., es una empresa multinacional mexicana dedicada a la operación de marcas de restauración franquiciadas. Con presencia en América Latina y Europa, se ha consolidado como uno de los principales operadores del sector, gestionando cadenas tan conocidas como Starbucks, Burger King, Domino's Pizza, Vips o Foster's Hollywood [1]. En España, su actividad se articula a través de Food Service Project, que engloba establecimientos de restauración rápida, cafeterías y restaurantes de comida casual.¹

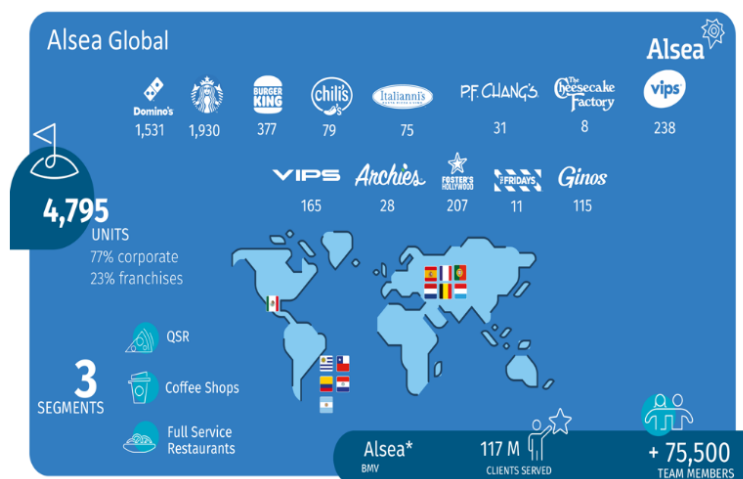


Figura 1.1: Mapa de presencia internacional de AESLA (Food Service Project). Fuente: elaboración propia a partir de [1].

La magnitud de esta operación internacional hace que AESLA maneje diariamente un volumen muy elevado de información relativa a ventas, operaciones, proyectos de innovación,

¹ Nota: El nombre del cliente ha sido modificado por motivos de confidencialidad.

indicadores de desempeño y actividades internas. En este contexto, disponer de datos fiables, centralizados y fácilmente accesibles resulta fundamental para evaluar el rendimiento de las franquicias, identificar posibles desviaciones y tomar decisiones estratégicas fundamentadas. Sin herramientas de análisis adecuadas, la gestión de este conocimiento se vuelve compleja y poco eficiente.

Para dar respuesta a esta necesidad, GBTEC Software AG, compañía alemana especializada en soluciones de gestión de procesos de negocio ([Business Process Management \(BPM\)](#)), arquitectura empresarial ([Enterprise Architecture Management \(EAM\)](#)) y gobierno, riesgo y cumplimiento ([Governance, Risk and Compliance \(GRC\)](#)), ofrece la *BIC Platform*, un ecosistema de software orientado al modelado, ejecución y análisis de procesos empresariales [3, 4]. En este contexto, se desarrolló para AESLA un módulo específico denominado *Report Service*, cuya finalidad es transformar la información procedente de los procesos ejecutados en la plataforma en informes ejecutivos descargables en formato Excel.

La *BIC Platform* integra distintos módulos que cubren diferentes etapas del ciclo de vida de los procesos empresariales. Entre ellos se encuentran *Process Design*, utilizado para modelar y documentar procesos mediante diagramas BPMN, y *Process Execution*, orientado a la ejecución y automatización de dichos procesos y a la gestión de sus instancias, tareas y variables asociadas [2, 4]. El *Report Service* utiliza información procedente de *Process Execution*, complementada con consultas a [Elasticsearch](#), para generar libros Excel que contienen [KPI](#), [Retroplanning](#), [Heatmap](#) y otras métricas definidas de acuerdo con los requisitos del proyecto.

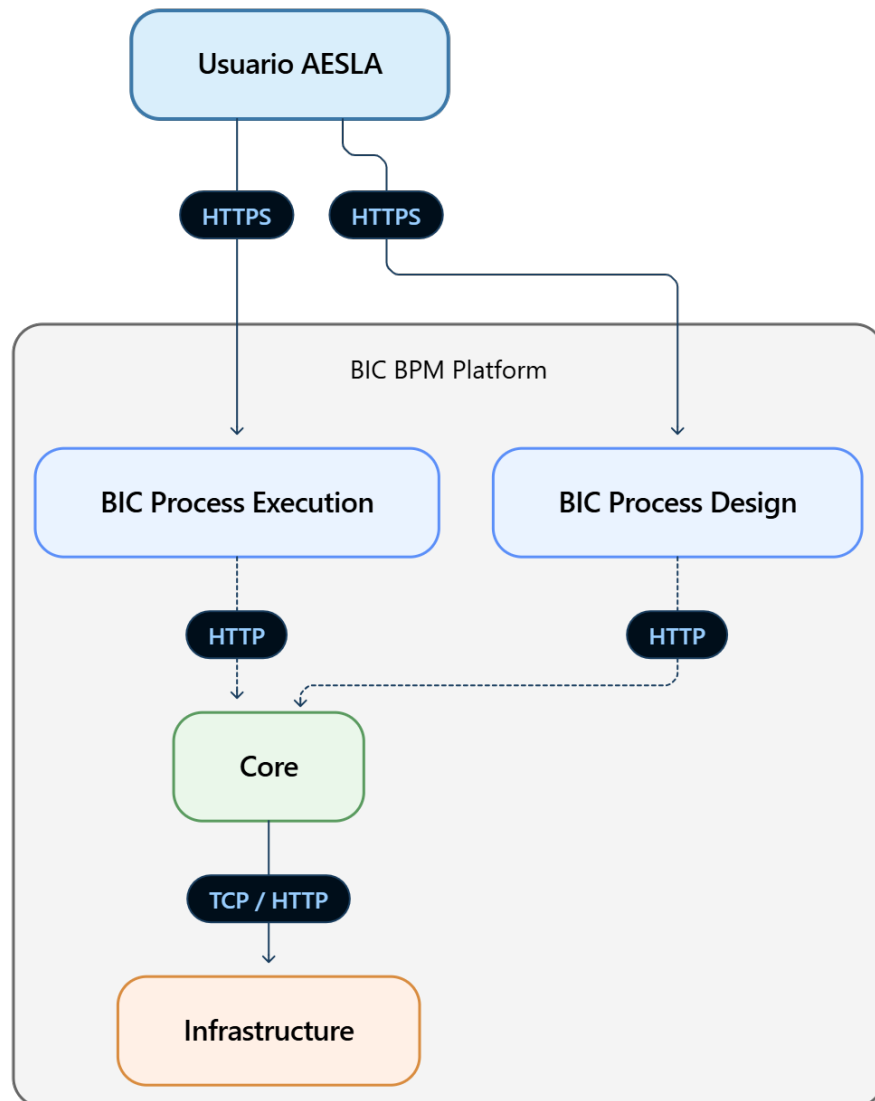


Figura 1.2: Visión general de la BIC BPM Platform y de los módulos principales con los que interactúa el usuario. Fuente: elaboración propia.

La figura 1.2 presenta una vista simplificada de la plataforma BIC y de los principales módulos implicados. El usuario accede mediante HTTPS a *BIC Process Design* y *BIC Process Execution*, mientras que ambos módulos se comunican con el núcleo de la plataforma, que a su vez interactúa con la infraestructura subyacente.

Para facilitar la comprensión del papel de *BIC Process Design* dentro de la plataforma, la figura 1.3 muestra un ejemplo sencillo de un proceso de negocio modelado mediante BPMN. Este tipo de diagramas permite representar gráficamente el flujo de un proceso mediante

eventos, actividades y decisiones, proporcionando una visión estructurada de las distintas etapas que lo componen.

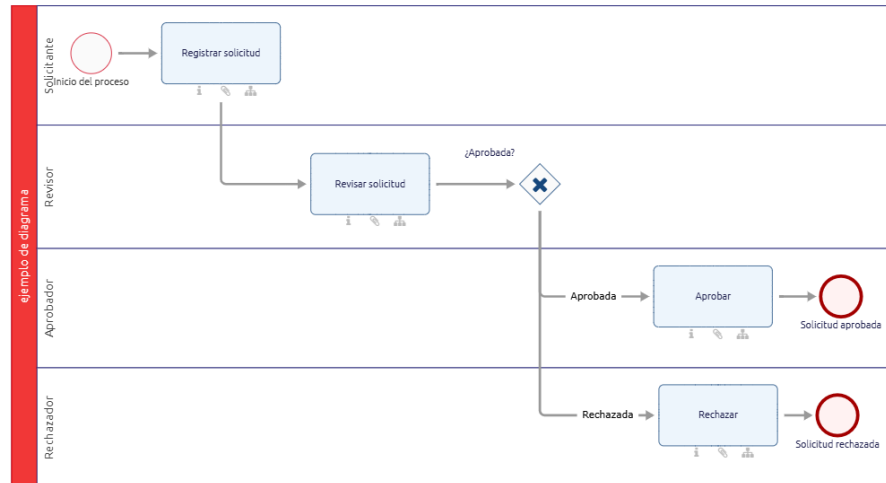


Figura 1.3: Ejemplo de un proceso BPMN sencillo modelado en BIC Process Design. Fuente: elaboración propia a partir de [2].

Los modelos definidos en *Process Design* sirven como representación de los procesos de negocio de la organización. Cuando estos procesos se trasladan al ámbito de ejecución, *Process Execution* permite gestionar sus instancias, tareas y variables asociadas. Parte de esta información constituye posteriormente la fuente de datos utilizada por el *Report Service* para elaborar los informes solicitados por AESLA.

El Report Service original ofrecía una solución sencilla y funcional: la interfaz permitía seleccionar el tipo de informe (KPI o Producto), la categoría (Innovación o Alternativa), una o varias franquicias y un rango de fechas. Con esta información, el backend recuperaba los datos correspondientes, generaba un libro de Excel con múltiples hojas y lo devolvía al usuario para su descarga local.

Cada combinación produce un libro Excel con múltiples hojas que cubren distintos requisitos y métricas personalizadas solicitadas por el cliente. A continuación se muestran dos ejemplos representativos que ilustran la estructura y contenido de estos informes:

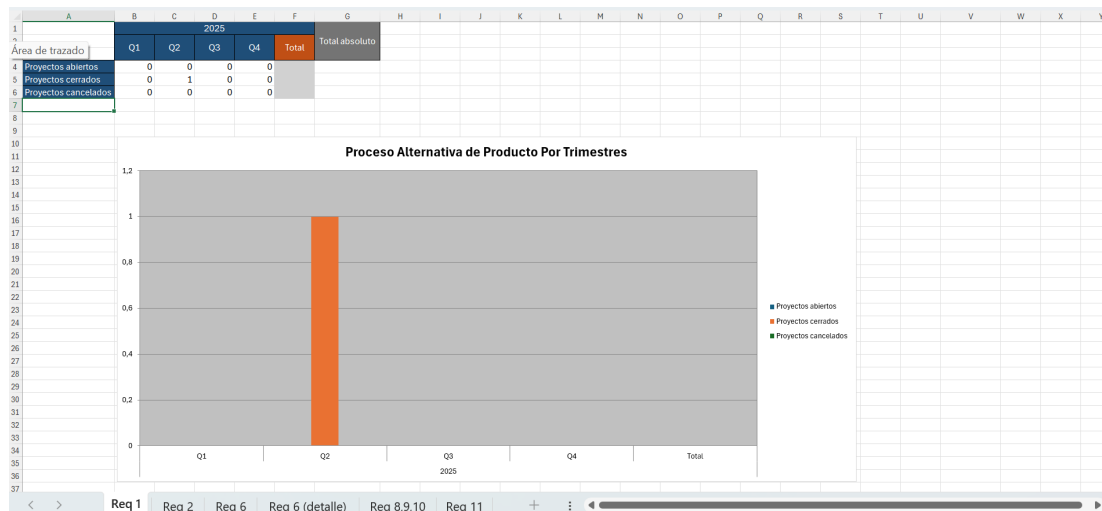


Figura 1.4: Informe de tipo KPI con categoría de proceso Alternativa (todas las franquicias, sin rango de fechas). Fuente: resultado de la ejecución del Report Service.

La anterior figura muestra la hoja del Requisito 1, que presenta un **Histograma** con los procesos de alternativa de producto por trimestres, diferenciando proyectos abiertos, cerrados y cancelados, además de los totales y el total absoluto en 2025. En la parte inferior se observan las pestañas de otras hojas del libro, correspondientes a requisitos adicionales personalizados solicitados por el cliente.

A	B	C	D	E	F	G	H	I
Marca	Estado	Proyecto	Actividad	Responsable	Fecha de comienzo inicial	Fecha de comienzo actual	Fecha de finalización inicial	Fecha de finalización actual
Foster's Hollywood	ACTIVO	test_ana_6	Trabajosddp	DDP	04-jun-25		19-jun-25	
Foster's Hollywood	ACTIVO	test_ana_6	DDP2	DDP	19-jun-25		03-ago-25	
Starbucks España	ACTIVO	test_bruno	Estimacionventa	MKT	25-may-25		01-jun-25	
Starbucks España	ACTIVO	test_bruno	Comunicacionventa	LOGISTICA	01-jun-25		08-jun-25	
Starbucks España	ACTIVO	test_bruno	Calidadconsumo	GESTION DE PRODUCTO	08-jun-25		08-jun-25	
Starbucks España	ACTIVO	test_bruno	Formacionvta	QA-ANA	08-jun-25		22-jun-25	
Starbucks España	ACTIVO	test_bruno	Productomarcas	LOGISTICA	22-jun-25		20-jul-25	
Starbucks Holand	ACTIVO	test_bruno_2	DDP2	DDP	19-abr-25		03-jun-25	
Starbucks Holand	ACTIVO	test_bruno_2	Muestras	GESTION DE PRODUCTO	03-jun-25		10-jun-25	
Starbucks Holand	ACTIVO	test_bruno_2	Catso	DDP	10-jun-25		17-jun-25	
Starbucks Holand	ACTIVO	test_bruno_2	DecisionMarca	MKT	17-jun-25		24-jun-25	
Starbucks Portugal	ACTIVO	test_bruno_3	Estimacionventa	MKT	24-jun-25		01-jul-25	
Starbucks Portugal	ACTIVO	test_bruno_3	Focus	C_INTELIGENCE	14-may-25		04-jun-25	
Starbucks Portugal	ACTIVO	test_bruno_3	Maguetas	GESTION DE PRODUCTO	04-jun-25		11-jun-25	
Starbucks Portugal	ACTIVO	test_bruno_3	Catso	DDP	11-jun-25		19-jun-25	
Starbucks Portugal	ACTIVO	test_bruno_3	DecisionMarca	MKT	19-jun-25		25-jun-25	
Starbucks Portugal	ACTIVO	test_bruno_3	Internacional	MKT	25-jun-25		02-jul-25	

Figura 1.5: Informe de tipo Producto con categoría de proceso Innovación (todas las franquicias, sin rango de fechas). Fuente: resultado de la ejecución del Report Service.

La anterior figura muestra la hoja del heatmap actual, que representa para cada marca el estado de sus proyectos junto con las actividades, responsables que las llevan a cabo y fechas previstas. En la parte inferior se pueden observar las pestañas de otras hojas del libro,

incluyendo los retroplannings y una hoja de comparativa, que forman parte del conjunto de métricas y análisis solicitados por AESLA.

Estos ejemplos muestran la riqueza informativa y el nivel de personalización de los informes generados por el Report Service, diseñados para cubrir las necesidades analíticas específicas de AESLA en la gestión de sus procesos de innovación y alternativa de producto.

Sin embargo, con el paso del tiempo surgieron limitaciones notables. El sistema carecía de un almacenamiento histórico de informes, lo que obligaba a los usuarios a regenerarlos cada vez que deseaban acceder a información pasada. Además, se detectaron problemas de duplicación de código en el backend, ausencia de procesamiento asíncrono (lo que generaba bloqueos en la interfaz durante la creación de los informes) y una experiencia de usuario que no reflejaba plenamente las necesidades de la organización. Estas carencias constituyeron la motivación principal para emprender un proceso de mejora y modernización del Report Service.

1.2 Objetivos

El presente Trabajo de Fin de Grado documenta, analiza y evalúa las mejoras introducidas en el Report Service de AESLA durante mis prácticas en GBTEC. En concreto, se plantearon los siguientes objetivos:

- Ampliar la funcionalidad del servicio con almacenamiento histórico en base de datos.
- Incrementar la mantenibilidad y calidad del código, reduciendo duplicaciones y aumentando la cobertura de pruebas.
- Optimizar la experiencia de usuario en la interfaz de generación de informes y acceso al historial.
- Introducir procesamiento asíncrono con [Spring Batch](#), evitando bloqueos en la interfaz.
- Añadir control de duplicidad para evitar regeneraciones innecesarias.
- Permitir la redescarga de informes desde el historial.

Con ello, se pretende transformar el Report Service en una solución completa que cubra tanto la generación como la gestión del ciclo de vida completo de los informes ejecutivos.

1.3 Aportación personal al proyecto

El Report Service era una aplicación existente antes del inicio de este Trabajo de Fin de Grado. Por tanto, el alcance del proyecto no consistió en desarrollar desde cero la lógica completa de generación de informes, sino en analizar la solución existente e implementar un conjunto de mejoras funcionales y técnicas sobre ella.

La tabla 1.1 diferencia de forma resumida las capacidades heredadas del sistema y las principales aportaciones realizadas durante el proyecto.

Tabla 1.1: Principales aportaciones realizadas durante el proyecto.

Área	Situación previa	Aportación realizada
Generación de informes	Generación y descarga inmediata del fichero	Integración del procesamiento asíncrono y seguimiento de la generación
Persistencia	No disponible	Incorporación de PostgreSQL, entidad, repositorio y servicio de persistencia
Migraciones	No disponible	Integración de Flyway y creación versionada del esquema y de las tablas necesarias
Historial	No disponible	Implementación de consulta paginada y recuperación de informes almacenados
Duplicidad	No disponible	Implementación de la detección de ejecuciones equivalentes recientes mediante metadatos de Spring Batch
Frontend	Interfaz centrada en la generación	Incorporación de mensajes de estado y página de historial
Calidad	Cobertura y duplicación por debajo de los objetivos del proyecto	Ampliación de pruebas, refactorización y reducción de duplicación
Dependencias	Existían incidencias y vulnerabilidades en el frontend	Estandarización del gestor de paquetes y mitigación de dependencias vulnerables

Fundamentos tecnológicos

2.1 Estado del arte

Para la elaboración del estado del arte se han analizado distintas soluciones disponibles en el mercado relacionadas con la generación de informes empresariales y cuadros de mando, evaluando sus ventajas y limitaciones. El objetivo de este análisis es comprender cómo se abordan actualmente estas necesidades en el sector y en qué medida las herramientas existentes inspiran o contrastan con el desarrollo del módulo Report Service en AESLA.

Microsoft Power BI es una plataforma de inteligencia de negocio orientada al análisis y visualización de datos. Permite conectarse a distintas fuentes de información y construir informes y cuadros de mando interactivos. También ofrece mecanismos de exportación de datos, aunque su enfoque principal se encuentra en la exploración visual y el análisis mediante dashboards, más que en la generación programática de libros Excel con una estructura completamente personalizada [5].

Tableau es una plataforma orientada al análisis visual de datos y a la creación de visualizaciones y cuadros de mando interactivos. Permite integrar diferentes fuentes de información y compartir los resultados dentro de entornos corporativos. Aunque dispone de opciones de exportación, su enfoque principal se centra en la visualización y exploración interactiva de la información, por lo que difiere del modelo de generación programática de informes utilizado en este proyecto [6].

Qlik Sense es una solución de analítica y visualización de datos que permite explorar información procedente de diferentes fuentes mediante su modelo asociativo. Proporciona funcionalidades para la creación de dashboards, análisis interactivo y exportación de información. Al igual que otras plataformas de inteligencia de negocio, su objetivo principal no es

la construcción programática de libros Excel adaptados a una lógica de negocio específica [7].

JasperReports es una solución orientada a la generación programática de informes a partir de plantillas y fuentes de datos configurables [8]. Este enfoque se aproxima más al problema tratado en este TFG, aunque requiere desarrollar y mantener la lógica específica de integración y generación para cada dominio de negocio.

Apache POI es una biblioteca de código abierto de la Apache Software Foundation que proporciona APIs Java para la lectura, creación y modificación de documentos de Microsoft Office. En el ámbito de las hojas de cálculo, ofrece soporte para los formatos XLS y XLSX mediante las APIs HSSF, XSSF y SXSSF [9]. A diferencia de las plataformas de inteligencia de negocio, no proporciona una solución completa de reporting orientada al usuario final, sino un conjunto de herramientas de programación sobre las que el desarrollador debe implementar la estructura, lógica y formato de cada informe. Por ello, constituye una alternativa técnica más próxima al enfoque programático utilizado por el Report Service.

La comparación se resume en la tabla 2.1. Las valoraciones son cualitativas y se refieren a las características descritas en la documentación oficial de cada solución; no constituyen una prueba de rendimiento entre productos.

Tabla 2.1: Comparación cualitativa de soluciones relacionadas con el análisis y la generación de informes.

Solución	Enfoque principal	Soporte Excel	Ex- cel	Generación de Excel personalizado	Integración con BIC
Power BI [5]	Inteligencia de negocio y visualización	Sí		Principalmente exportación de datos y visualizaciones	No específica
Tableau [6]	Análisis y visualización interactiva	Sí		Principalmente exportación de datos o vistas	No específica
Qlik Sense [7]	Analítica y exploración de datos	Sí		Principalmente exportación de datos de visualizaciones	No específica
JasperReports [8]	Generación programática de informes	Sí		Sí, mediante plantillas y lógica de generación	Requiere integración específica
Apache POI [9]	Manipulación programática de documentos Office	Sí, lectura y escritura		Sí, mediante código Java	Requiere desarrollo específico
Report Service	Informes específicos sobre procesos BIC	Sí		Sí, adaptada a los requisitos de AESLA	Integración directa con la plataforma

Las soluciones analizadas responden a necesidades diferentes. Power BI, Tableau y Qlik Sense están orientadas principalmente al análisis visual y a los cuadros de mando, aunque permiten exportar los datos mostrados a formatos compatibles con Excel. JasperReports se aproxima más al problema tratado en este proyecto, al proporcionar un motor programático de generación de informes basado en plantillas.

El Report Service presenta, sin embargo, un ámbito más específico: se integra directamente con los datos y procesos de la plataforma BIC y genera libros Excel cuya estructura, hojas, métricas y reglas de negocio responden a los requisitos concretos de AESLA. Por tanto, no pretende sustituir a una plataforma generalista de inteligencia de negocio, sino resolver una necesidad de reporting especializada dentro del ecosistema existente.

2.2 Tecnologías utilizadas

El desarrollo del Report Service se apoyó en un conjunto de tecnologías que abarcan tanto el cliente web como el servidor y la base de datos, junto con utilidades complementarias para despliegue, monitorización y calidad.

2.2.1 Servidor web

- **Spring Boot:** framework de desarrollo en Java que permite crear aplicaciones web y microservicios de forma rápida y estructurada. Aporta configuración automática, inyección de dependencias y un ecosistema rico de librerías [10].
- **Spring Batch:** módulo de Spring orientado al procesamiento por lotes. Se utilizó para implementar la generación asíncrona de informes, gestionando *jobs* y *steps* de manera controlada [11].
- **Hibernate / JPA:** framework de mapeo objeto-relacional que simplifica el acceso a la base de datos mediante entidades Java, evitando tener que escribir consultas SQL manuales [12].
- **Aspose Cells:** es una librería de generación y manipulación de Excel utilizada para dar formato, estilos y estructura a los informes [13].
- **Elasticsearch:** motor de búsqueda utilizado como fuente de datos adicional, especialmente para recuperar información de instancias de procesos con consultas JSON flexibles [14].
- **Maven:** herramienta de gestión y automatización de proyectos Java, empleada para manejar dependencias, construir el proyecto y ejecutar tareas comunes [15].

2.2.2 Cliente web

- **Angular:** framework de desarrollo basado en TypeScript utilizado para construir la interfaz del usuario. Permite crear componentes modulares, mantener un enrutamiento dinámico y gestionar formularios y servicios de forma escalable [16].
- **Yarn:** gestor de dependencias para proyectos JavaScript que garantiza instalaciones reproducibles y seguras. Fue la herramienta principal para administrar librerías del frontend y realizar auditorías de seguridad.
- **Node.js:** entorno de ejecución necesario para compilar y ejecutar Angular en local, así como para el servidor de desarrollo.

- **TypeScript:** lenguaje de programación que extiende JavaScript con tipado estático y características avanzadas, facilitando el desarrollo de aplicaciones robustas y mantenibles.

2.2.3 Base de datos

- **PostgreSQL:** sistema de gestión de bases de datos relacional, robusto y de código abierto, empleado para almacenar los informes generados y sus metadatos [17].
- **Flyway:** herramienta de migraciones que asegura el versionado y la coherencia del esquema de base de datos en todos los entornos (desarrollo, pruebas y producción). Se utilizó para crear y mantener el esquema *aesla_reports* y su tabla principal *report* [18].
- **Testcontainers:** framework de testing que permite levantar contenedores efímeros de PostgreSQL durante la ejecución de pruebas, garantizando un entorno controlado y reproducible [19].

2.2.4 Infraestructura y Despliegue

- **Docker:** plataforma de contenedores utilizada para empaquetar tanto el backend como el frontend y garantizar despliegues reproducibles en diferentes entornos [20].
- **Git:** sistema de control de versiones empleado en combinación con ramas de funcionalidad y *Pull Requests*, asegurando trazabilidad y revisiones de código antes de su integración.

2.2.5 Herramientas de apoyo

- **Gestión de trabajo:** Jira se utilizó junto con un tablero Kanban para organizar elementos de tipo *Epic*, historias de usuario y subtareas, así como para gestionar prioridades y realizar el seguimiento visual del progreso de cada ticket [21].
- **Control de versiones y hosting:** Git para control de versiones y GitHub como repositorio remoto centralizado donde se alojaba el código fuente del proyecto, facilitando la colaboración mediante Pull Requests y revisiones de código.
- **Flujo de trabajo Git:** Gitflow [22] se adoptó como convención de ramas (feature/fix/hotfix) para mantener estabilidad en las ramas de producción y organizar el desarrollo de forma estructurada.
- **Integración continua / CI & CD:** GitHub Actions se configuró con acciones personalizadas para automatizar el build del proyecto, ejecutar pruebas unitarias e integradas, y

realizar despliegues [23]. Los pipelines se integraron con SonarQube (SonarCloud) para validar métricas de calidad de código ([Coverage](#), [Duplicación de código](#), [Code Smells](#)) antes de aceptar cada [PR](#) [24].

- **Registro y gestión de artefactos:** [Quay](#) se empleó como registry principal para almacenar y versionar las imágenes Docker de los entornos Ansible de prueba, mientras que [JFrog Artifactory](#) sirvió como repositorio de artefactos binarios y dependencias Maven/NPM, facilitando la distribución controlada de versiones.
- **Orquestación de despliegue y automatización:** [Ansible](#) (gestionado mediante [AWX](#)) permitió levantar entornos de prueba y automatizar despliegues en [QA](#).
- **Entornos de desarrollo:** IntelliJ IDEA para backend (Java/Spring) y Visual Studio Code para frontend (Angular/TypeScript).
- **Pruebas y depuración:** [Postman](#) para pruebas de [API](#) y Google Chrome para pruebas manuales en entorno local.
- **Calidad de código:** [SonarCloud](#) se integró en los pipelines de [CI](#) para realizar análisis estático de código ([HTML](#), [CSS](#), TypeScript, Java), medir cobertura de pruebas, detectar duplicaciones y controlar la deuda técnica, garantizando que cada cambio cumpliera con los umbrales de calidad definidos por la empresa.

Metodología y planificación

3.1 Metodología de desarrollo

El proyecto se llevó a cabo siguiendo una metodología ágil [25] basada en iteraciones cortas y gestión Kanban [26] en [Jira](#). Este enfoque resultó adecuado por la naturaleza incremental de las mejoras del Report Service, la necesidad de integrar cambios en backend y frontend con validación continua, y la conveniencia de priorizar incidencias, tareas técnicas y funcionalidades nuevas. Este tablero Kanban se configuró con el objetivo de permitir visualizar el flujo de trabajo completo, con estados definidos (*To Do* → *Task Breakdown* → *In Progress* → *Ready for QA/PR* → *QA/PR OK* → *Done*) para cada tarea, facilitando el seguimiento de prioridades y la identificación de bloqueos.

Las características principales del enfoque fueron:

- **Iterativo e incremental:** cada conjunto de cambios (feature/refactor/fix/chore) se gestionó como un elemento de trabajo en el tablero, produciendo un incremento funcional verificable (código, tests y PR).
- **Retroalimentación continua:** reuniones diarias con el equipo para sincronizar prioridades, desbloquear tareas y validar entregables; revisión de PRs con comentarios técnicos.
- **Flexibilidad:** replanificación rápida en función de prioridades del [EPIC](#) y de incidencias no previstas inicialmente que surgieron durante el desarrollo (p. ej., unificación de estructuras de peticiones entre tipos de informe, compatibilidad de parámetros opcionales como fechas nulas, detección de duplicidades), las cuales se integraron de manera ágil en las tareas correspondientes.

3.1.1 Adaptación al proyecto

La incorporación al proyecto siguió una fase breve de formación, alineada con las tecnologías clave:

- **Formación inicial:** cursos introductorios de Spring Batch, [Docker](#), patrones de diseño en Java y Angular (priorizando primero el backend), proporcionados mediante la suscripción de Udemy contratada por la empresa.
- **Aterrizaje en Jira:** asignación al [EPIC Improvements for AESLA Project in 2025 Q3 & Q4](#), que agrupaba los work items relacionados con la evolución del Report Service. La mayoría se clasificaron como *Task*, aunque también se gestionaron incidencias de tipo *Bug* y una historia de usuario de mayor alcance para introducir la generación asíncrona.
- **Flujo de trabajo:**
 - Rama por tarea (feature/fix branch) en Git.
 - Commits atómicos y descriptivos.
 - Pull Request hacia *development*, con code review y checks automáticos de SonarCloud y build de pull/push.
 - Cierre de la tarea al completar [PR](#) y [QA](#).
 - Uso de una misma *fix version* del Report Service para agrupar los cambios que debían publicarse conjuntamente.

El orden de adaptación técnica fue: backend → datos → frontend, permitiendo una integración progresiva. Para cada work item se creó una rama independiente en Git, identificada con el número de la tarea, donde se realizaron los commits correspondientes. Una vez finalizado el desarrollo y superada la revisión de código y la validación de QA, se aprobaba el Pull Request y se integraban los cambios en la rama *development* del Report Service.

3.2 Planificación y seguimiento

El proyecto se desarrolló durante los meses de julio, agosto y septiembre. La dedicación al trabajo fue de 7 horas diarias durante julio y agosto, y de 6 horas diarias durante septiembre, al tener que compaginar las prácticas con el inicio del curso universitario.

La planificación del proyecto se estructuró en fases de trabajo, en lugar de *sprints* formales, de acuerdo con la metodología ágil basada en Kanban utilizada por el equipo. Esta forma de organización permitió adaptar el ritmo de trabajo y la prioridad de las tareas a las necesidades reales del proyecto, manteniendo un seguimiento continuo del estado de cada *work item*.

Con el objetivo de reflejar tanto la previsión inicial como la evolución efectiva del proyecto, se presentan dos diagramas de Gantt: uno correspondiente a la **planificación inicial** y otro al **seguimiento real**. La comparación entre ambos permite identificar de forma visual las fases que se prolongaron más de lo previsto inicialmente.

3.2.1 Planificación inicial

La planificación inicial se definió de forma orientativa al comienzo del proyecto. En ella se preveía una primera fase breve de incorporación y formación, seguida por tareas de refactorización y limpieza del código. Posteriormente se abordaría la unificación de la gestión de informes y la evolución del backend como núcleo principal del trabajo. Finalmente, la integración del frontend se reservaría para septiembre, dejando la última semana para las tareas de cierre, liberación de versión y documentación.

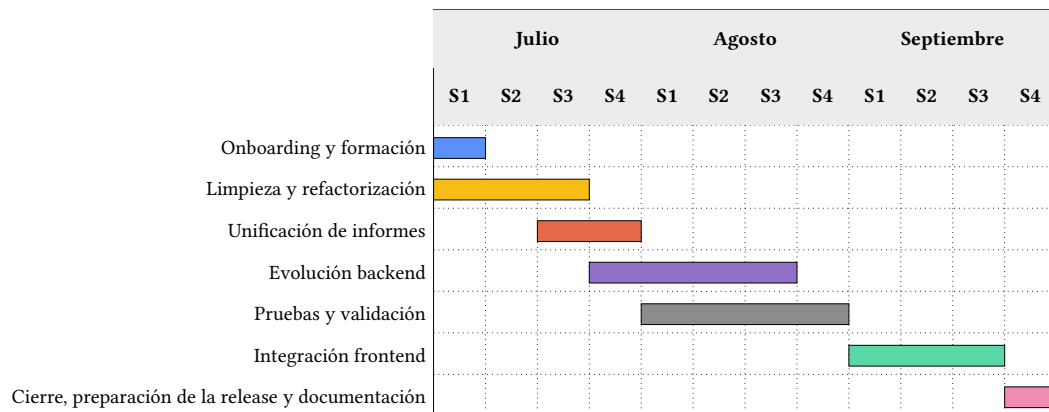


Figura 3.1: Diagrama de Gantt de la planificación inicial del proyecto.

Como puede observarse, la planificación inicial planteaba una secuencia relativamente ordenada: una primera fase de adaptación durante la primera semana de julio, una etapa de refactorización durante el resto del mes, la transición hacia la unificación del servicio entre finales de julio y comienzos de agosto, el desarrollo del backend como fase central del proyecto, y finalmente la integración del frontend y el cierre durante septiembre.

3.2.2 Seguimiento real

Durante la ejecución del proyecto, la planificación inicial tuvo que adaptarse a las necesidades que fueron surgiendo, en coherencia con la metodología ágil empleada. Algunas tareas previstas resultaron más complejas de lo estimado inicialmente, especialmente aquellas relacionadas con la unificación de la gestión de informes y la evolución del backend.

La unificación de la gestión de informes se prolongó más de lo previsto porque implicó

homogeneizar controladores, endpoints y DTOs, así como normalizar el tratamiento de parámetros opcionales y reorganizar la estructura interna de paquetes. Del mismo modo, la evolución del backend también requirió más tiempo, al incorporar persistencia de informes, endpoints de consulta y descarga, generación asíncrona con Spring Batch, control de duplicidad, registro de métricas y ajustes relacionados con la inicialización de Flyway y de las tablas asociadas a Spring Batch.

Además, las pruebas y validaciones no se limitaron a una fase puntual, sino que se mantuvieron a lo largo de buena parte del desarrollo, en forma de pruebas unitarias, integradas, revisión de *Pull Requests* y validación en QA. Como consecuencia, varias actividades se solaparon parcialmente sin alterar la fecha final de cierre del proyecto.

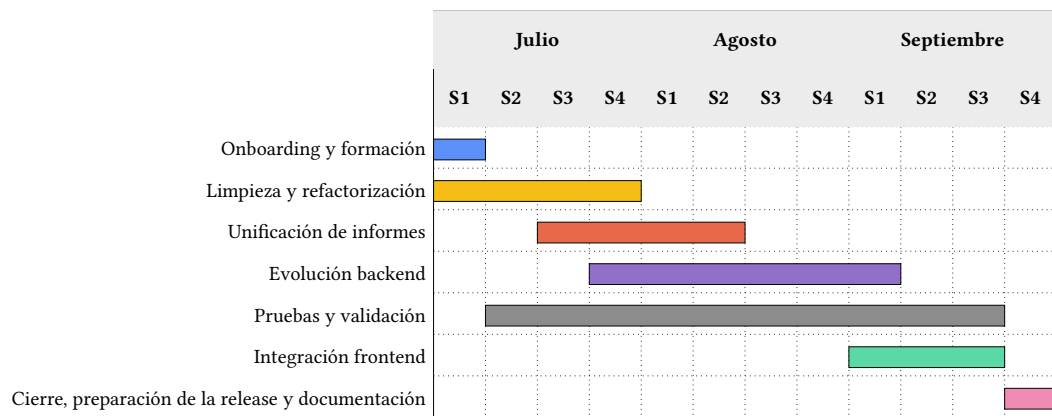


Figura 3.2: Diagrama de Gantt correspondiente al seguimiento real del proyecto.

Tabla 3.1: Comparación entre la planificación inicial y el seguimiento real del proyecto.

Fase	Plan inicial	Seguimiento real	Motivo principal de la desviación
Onboarding y formación	S1	S1	Sin desviaciones relevantes.
Limpieza y refactorización	S1–S3	S1–S4	La revisión del código existente reveló más lógica duplicada y dependencias entre componentes de las previstas inicialmente.
Unificación de informes	S3–S4	S3–S6	Fue necesario homogeneizar DTO, parámetros, endpoints y tratamiento de fechas y franquicias entre los distintos tipos de informe.
Evolución backend	S4–S7	S4–S9	La incorporación de persistencia, Spring Batch, control de duplicidad y migraciones requirió más trabajo de integración del estimado.
Pruebas y validación	S5–S8	S2–S11	Las pruebas se realizaron de forma transversal durante el desarrollo y no únicamente como una fase final.
Integración frontend	S9–S11	S9–S11	Se mantuvo aproximadamente dentro de la planificación inicial.
Cierre y documentación	S12	S12	Sin desviación sobre la fecha final prevista.

Como puede observarse en la tabla 3.1, las principales desviaciones se concentraron en la unificación de la gestión de informes y en la evolución del backend. No obstante, el solapamiento de actividades permitido por el flujo Kanban hizo posible absorber estas desviaciones sin modificar la fecha final del proyecto.

Capítulo 4

Análisis

4.1 Situación inicial y alcance

Antes de abordar la modernización del Report Service fue necesario analizar el comportamiento de la solución existente y delimitar los problemas que debían resolverse. La versión inicial permitía generar informes Excel a partir de los datos disponibles en BIC Process Execution y Elasticsearch, pero el flujo se centraba únicamente en la generación y descarga inmediata del fichero.

La interfaz original constaba de una única página desde la que el usuario podía seleccionar el tipo de informe, la categoría, una o varias franquicias y, cuando procedía, un rango de fechas. Una vez enviada la solicitud, el backend procesaba la información y construía el workbook correspondiente mediante Aspose Cells.

Aunque este flujo cubría la necesidad básica de reporting, presentaba varias limitaciones. Los informes no se almacenaban de forma persistente, por lo que era necesario regenerarlos para volver a acceder a ellos. La generación se realizaba de forma síncrona, manteniendo al usuario a la espera durante el procesamiento. Tampoco existía un mecanismo específico para evitar solicitudes duplicadas ni una página desde la que consultar generaciones anteriores.

A partir de esta situación se definió el alcance de la modernización: incorporar persistencia e historial, desacoplar la generación del ciclo principal de la petición HTTP, introducir control de duplicidad, mejorar la información de estado mostrada al usuario y reforzar la mantenibilidad y calidad del código existente.

4.2 Actores y sistemas involucrados

En el sistema intervienen distintos actores y componentes que participan en la generación, almacenamiento y consulta de los informes ejecutivos:

- **Usuario AESLA:** empleado o analista encargado de generar y consultar los informes a través de la interfaz web. Puede lanzar nuevas generaciones, visualizar el historial y volver a descargar informes anteriores.
- **Aplicación web:** cliente desarrollado en Angular que permite la interacción del usuario con el sistema. Presenta formularios, mensajes de estado y botones de acción, y realiza el seguimiento periódico de las generaciones.
- **Servicio de reportes:** aplicación de servidor desarrollada con Spring Boot. Gestiona la lógica asociada a las solicitudes, coordina la generación asíncrona, controla la duplicidad y persiste los resultados.
- **Origen de datos de procesos:** BIC Process Execution y las fuentes disponibles en Elasticsearch, de donde se obtienen las variables necesarias para la composición de los informes.
- **Base de datos de informes:** sistema PostgreSQL donde se almacenan los informes generados junto con los metadatos necesarios para identificarlos y recuperarlos posteriormente.

Esta delimitación permite distinguir al actor principal de los sistemas que colaboran en la ejecución del caso de uso. La estructura interna concreta de estos componentes se desarrolla posteriormente en el capítulo 5.

4.3 Requisitos

4.3.1 Requisitos funcionales

Los requisitos funcionales definen el comportamiento observable que debe ofrecer la solución modernizada.

1. RF-01 - Generación de informes:

- El sistema debe permitir generar informes en formato Excel seleccionando el *tipo* (KPI o Producto), la *categoría* (Innovación o Alternativa), una o varias *franquicias* y, cuando proceda, un *rango de fechas* opcional.

- La generación debe ejecutarse de forma asíncrona, evitando que la interfaz permanezca bloqueada durante todo el procesamiento.
- La aplicación debe proporcionar información de estado durante el envío, generación, seguimiento y descarga.
- Si no existen datos para la combinación solicitada, el sistema debe finalizar el procesamiento sin generar un workbook vacío e informar de esta situación.

2. RF-02 - Control de duplicidad:

- Antes de iniciar una nueva generación, el sistema debe comprobar si existe una solicitud equivalente lanzada recientemente.
- La ventana temporal considerada para esta comprobación es de cinco minutos.
- En caso de duplicidad, no debe iniciarse un segundo job y se debe informar al usuario de la existencia de una solicitud reciente.
- La equivalencia de las solicitudes se determina a partir de los parámetros que identifican la generación.

3. RF-03 - Persistencia, descarga y redescarga:

- Los informes generados correctamente deben almacenarse de forma persistente.
- El contenido Excel y los metadatos asociados deben poder recuperarse posteriormente.
- El usuario debe poder descargar el informe al finalizar una generación válida.
- El usuario debe poder volver a descargar un informe ya almacenado sin necesidad de regenerarlo.

4. RF-04 - Historial de informes:

- El sistema debe disponer de una página de *Historial*.
- Para cada informe deben mostrarse los metadatos relevantes, como fecha de creación, tipo, categoría, franquicias y rango de fechas.
- Los informes deben presentarse ordenados de más reciente a más antiguo.
- El historial debe incorporar paginación mediante controles de navegación.

4.3.2 Requisitos no funcionales

Además de las funcionalidades anteriores, se identificaron propiedades de calidad que debían guiar el desarrollo:

- **RNF-01 - Usabilidad:** la interfaz debe utilizar controles y mensajes comprensibles que permitan identificar el estado de la solicitud y las acciones disponibles.
- **RNF-02 - Rendimiento percibido:** la generación de un informe no debe mantener bloqueada la interacción principal del cliente. Las consultas del historial deben realizarse mediante paginación para evitar recuperar de una sola vez todos los informes almacenados.
- **RNF-03 - Confiabilidad:** los informes completados deben quedar almacenados de forma persistente y la aplicación debe evitar generaciones duplicadas dentro de la ventana temporal definida.
- **RNF-04 - Mantenibilidad:** el código debe mantener una organización modular, separar responsabilidades y disponer de pruebas automatizadas sobre los componentes más relevantes.
- **RNF-05 - Capacidad de evolución:** la arquitectura debe permitir la ejecución asíncrona de trabajos y facilitar futuras mejoras de capacidad sin requerir cambios sustanciales en el flujo funcional.
- **RNF-06 - Seguridad:** la solución debe integrarse con los mecanismos de acceso existentes en la plataforma BIC y mantener controladas las dependencias utilizadas por el frontend y el backend.
- **RNF-07 - Observabilidad:** los puntos relevantes de generación deben disponer de registros que faciliten el seguimiento y diagnóstico de incidencias.
- **RNF-08 - Compatibilidad:** el cliente debe funcionar en los navegadores utilizados en el entorno corporativo y los ficheros producidos deben mantener el formato Excel requerido.

4.4 Historias de usuario

A partir de los requisitos se formularon las principales historias de usuario que guiaron el desarrollo funcional:

- **HU01 – Generar informe:** como usuario, quiero seleccionar el tipo, categoría, franquicias y rango de fechas para generar un informe Excel personalizado.
- **HU02 – Visualizar estados de generación:** como usuario, quiero recibir información sobre el estado de la solicitud para saber si la generación ha comenzado, continúa en proceso, ha finalizado o se ha producido una incidencia.

- **HU03 – Control de duplicidad:** como usuario, quiero que el sistema detecte solicitudes equivalentes recientes para evitar regeneraciones innecesarias.
- **HU04 – Descargar informe:** como usuario, quiero descargar el informe una vez finalizada correctamente la generación.
- **HU05 – Consultar historial:** como usuario, quiero acceder a un listado de los informes generados y consultar la información asociada a cada uno.
- **HU06 – Redescargar informe:** como usuario, quiero volver a descargar un informe anterior sin tener que generarlo de nuevo.

4.5 Casos de uso

La figura 4.1 representa las funcionalidades principales accesibles desde el punto de vista del usuario. El control de duplicidad, la comprobación de disponibilidad de datos y el seguimiento mediante *polling* se consideran comportamientos internos asociados al caso de uso de generación.

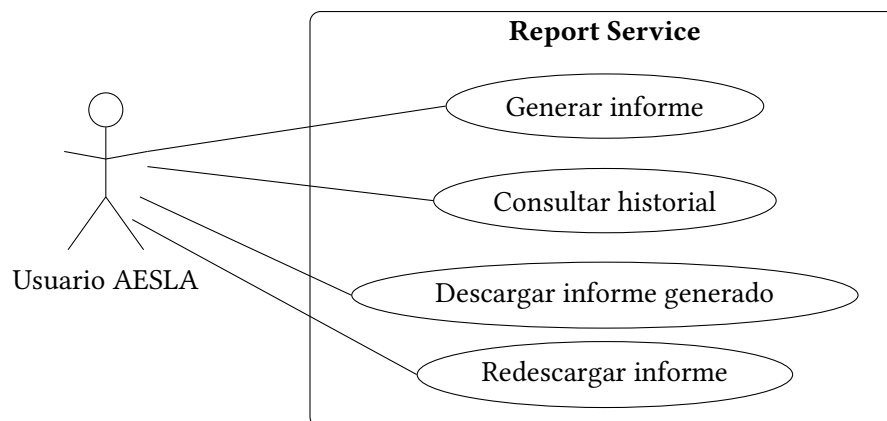


Figura 4.1: Diagrama de casos de uso principales del Report Service. Fuente: elaboración propia.

4.6 Interfaz original y limitaciones

La interfaz de usuario original del Report Service presentaba un diseño sencillo y funcional. El usuario disponía de una única página desde la que podía configurar los parámetros básicos del informe: tipo, categoría, franquicias y un rango de fechas opcional.

Una vez configurados estos parámetros, la solicitud se procesaba de forma síncrona. Durante ese tiempo la interfaz permanecía condicionada por la operación de generación y el usuario no disponía de un mecanismo específico para consultar posteriormente el resultado. Cuando la operación finalizaba, el fichero Excel se descargaba directamente en el navegador.

Las principales limitaciones detectadas fueron la ausencia de historial, la falta de persistencia de los informes, el procesamiento síncrono, la escasa información de estado durante la generación y la inexistencia de un control de duplicidad. Estas carencias sirvieron como punto de partida para el diseño de la solución descrito en el capítulo 5.

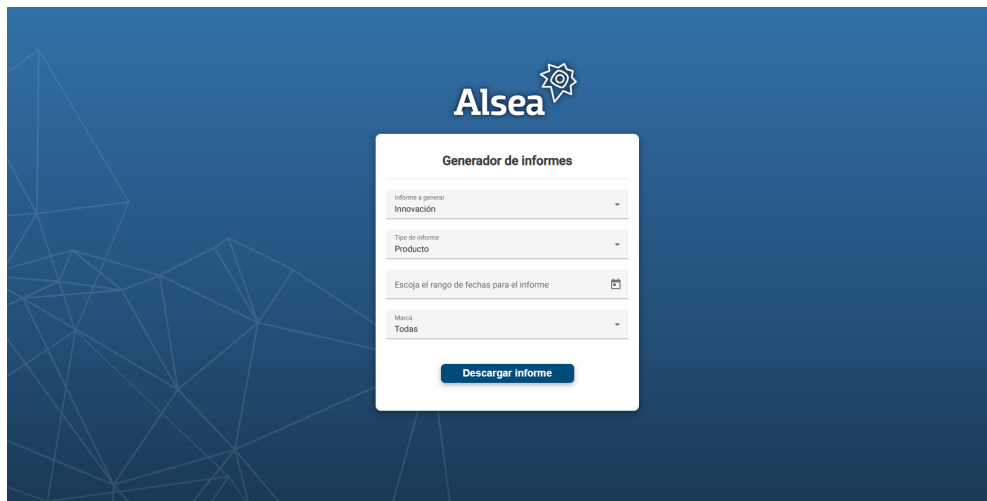


Figura 4.2: Interfaz original del Report Service antes de su modernización. Fuente: captura propia.

4.7 Trazabilidad entre limitaciones y requisitos

Las limitaciones identificadas durante el análisis de la solución original sirvieron como base para establecer los requisitos funcionales y no funcionales de la modernización. La tabla 4.1 resume esta relación y permite mantener la trazabilidad entre los problemas detectados y las decisiones posteriores de diseño.

Tabla 4.1: Relación entre las limitaciones iniciales y los requisitos derivados.

Limitación detectada	Impacto	Requisito derivado
Ausencia de persistencia	Los informes debían regenerarse para volver a acceder a ellos	RF-03 / RF-04
Generación síncrona	El usuario debía esperar durante el procesamiento del informe	RF-01 / RNF-02
Ausencia de control de duplicidad	Una misma configuración podía provocar generaciones redundantes	RF-02 / RNF-03
Ausencia de historial	No existía un mecanismo de consulta o recuperación posterior	RF-04
Código duplicado	Aumentaba el coste de mantenimiento y el riesgo de inconsistencias	RNF-04
Observabilidad limitada	Dificultaba el diagnóstico y seguimiento de incidencias	RNF-07

Diseño de la solución

5.1 Arquitectura tecnológica del sistema

En este capítulo se describe cómo se estructuró la solución para satisfacer los requisitos identificados en el capítulo 4. El objetivo es presentar las responsabilidades de cada componente y las decisiones de diseño adoptadas, dejando para el capítulo 6 los detalles concretos de archivos, migraciones, configuración y cambios realizados sobre el código.

5.1.1 Visión general de la arquitectura

El Report Service sigue una arquitectura cliente-servidor organizada en tres capas principales (presentación, lógica de aplicación y datos), complementadas por los componentes de infraestructura necesarios para la migración de base de datos, contenerización y despliegue.

- **Capa de presentación:** aplicación web desarrollada en Angular 15 que proporciona la interfaz necesaria para configurar una generación, consultar su estado, acceder al historial y descargar informes.
- **Capa de lógica de aplicación:** backend basado en Spring Boot 3 y Spring Batch encargado de recibir las peticiones, comprobar la duplicidad, coordinar el procesamiento asíncrono, seleccionar la lógica de generación y gestionar la persistencia.
- **Capa de datos:** PostgreSQL almacena los informes generados y sus metadatos, mientras que BIC Process Execution y Elasticsearch actúan como fuentes de información para construir los distintos tipos de workbook.

La tabla 5.1 resume los componentes principales y la responsabilidad asignada a cada uno.

Tabla 5.1: Componentes principales y responsabilidades del Report Service.

Componente	Responsabilidad principal
Cliente Angular	Recoger los parámetros, validar la entrada, mostrar el estado de la solicitud y permitir consultar y descargar informes.
<i>ReportBatchController</i>	Exponer las operaciones REST relacionadas con generación, seguimiento, consulta, descarga y eliminación de informes.
<i>ReportParamService</i>	Normalizar y preparar los parámetros recibidos por la API antes de utilizarlos en la generación o en las consultas.
<i>ReportLaunchService</i>	Coordinar las comprobaciones previas y determinar si una solicitud debe iniciarse, rechazarse por duplicidad o finalizar sin datos.
<i>ReportDuplicateUtils</i>	Detectar solicitudes equivalentes ejecutadas durante los últimos cinco minutos comparando los parámetros de negocio almacenados en los metadatos de Spring Batch.
<i>JobExplorer</i>	Proporcionar acceso a las instancias y ejecuciones registradas por Spring Batch para realizar la comprobación de duplicidad.
<i>GenerateReportTasklet</i>	Ejecutar la lógica asociada al step de generación y coordinar la creación del workbook correspondiente.
<i>ReportDataProbeService</i>	Comprobar la disponibilidad de información para la combinación de tipo, categoría, franquicias y fechas solicitada.
<i>ReportService</i>	Centralizar la persistencia, consulta, recuperación y eliminación de los informes almacenados.
<i>ReportRepository</i>	Encapsular mediante JPA el acceso a los informes almacenados en PostgreSQL.

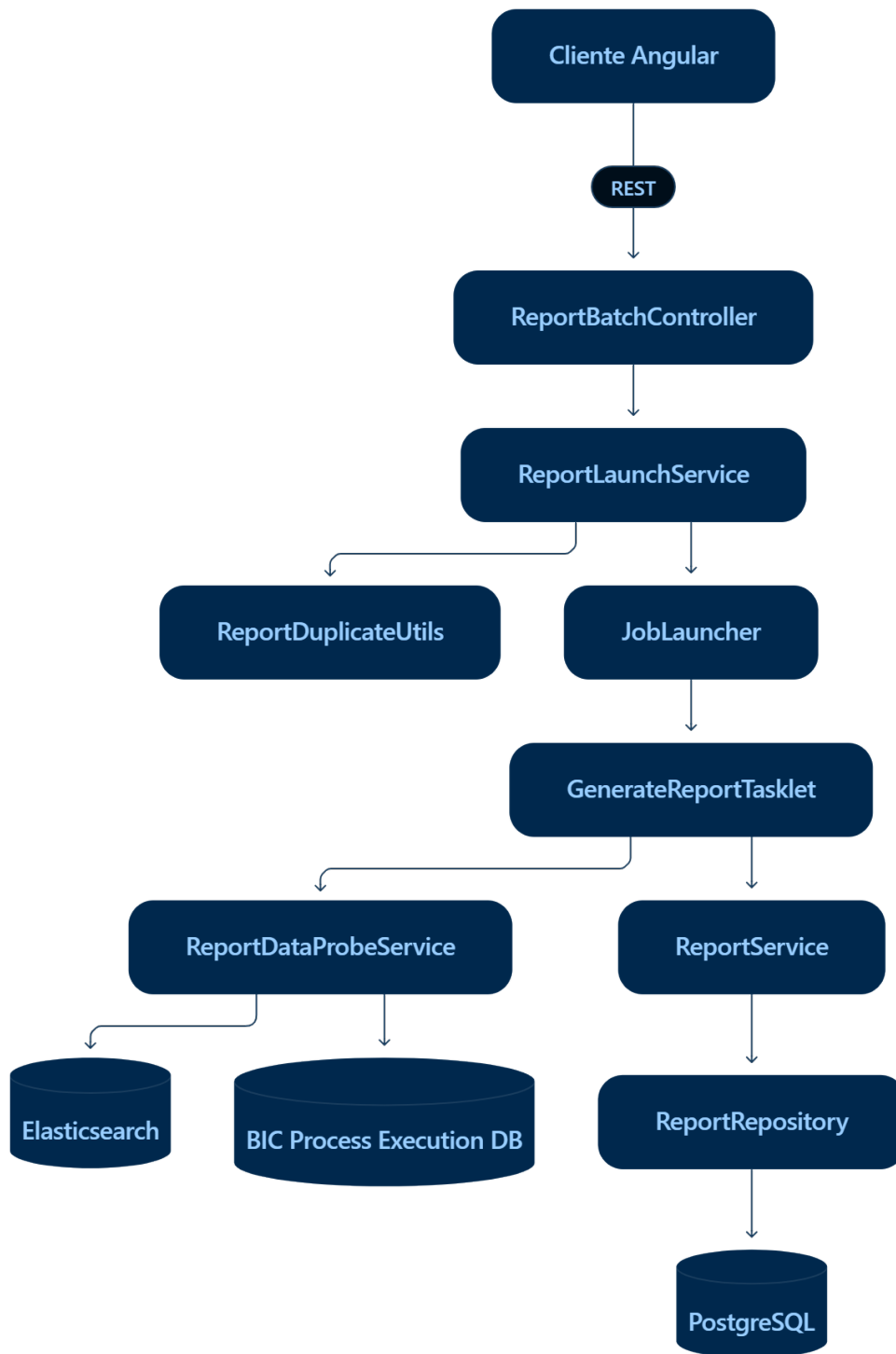


Figura 5.1: Arquitectura simplificada del flujo principal de generación de informes.

La figura 5.1 muestra el flujo principal. El cliente envía la solicitud al controlador REST,

que delega en *ReportLaunchService*. Antes de iniciar el procesamiento se comprueba la posible duplicidad mediante *ReportDuplicateUtils* y la información persistida. Si la solicitud puede procesarse, se lanza de forma asíncrona el job de Spring Batch.

Durante la ejecución, *GenerateReportTasklet* utiliza *ReportDataProbeService* para comprobar la disponibilidad de datos y selecciona el generador correspondiente al tipo y la categoría solicitados. Una vez construido el workbook, el almacenamiento se delega en *ReportService*, que utiliza *ReportRepository* para persistir el resultado en PostgreSQL.

La representación detallada, incluyendo componentes internos del frontend y backend, fuentes de datos e infraestructura, se incluye en el apéndice [A.1](#).

5.1.2 Comunicación entre componentes

La comunicación entre cliente y servidor se realiza mediante una [API REST](#) sobre [HTTP](#). Las solicitudes y respuestas estructuradas utilizan [JSON](#), mientras que la descarga del informe utiliza contenido binario.

Los principales recursos diseñados para esta interacción son:

- *POST /api/reports*: inicio de una nueva generación.
- *GET /api/reports/latest*: seguimiento periódico de la disponibilidad del informe.
- *GET /api/reports*: consulta paginada de informes almacenados.
- *GET /api/reports/{id}/download*: recuperación del contenido binario de un informe concreto.

El control de duplicidad y la comprobación de disponibilidad de datos se situaron en momentos diferentes del flujo. La duplicidad se evalúa antes de lanzar un nuevo job, mientras que la disponibilidad de datos se comprueba una vez iniciada la ejecución mediante *ReportDataProbeService.hasData()*. Si no existen datos, el step finaliza con el estado *NO_DATA* sin construir el workbook.

5.1.3 Justificación de la arquitectura elegida

Las principales decisiones de diseño responden a los siguientes criterios:

- **Modularidad**: cada componente concentra una responsabilidad específica y reduce el acoplamiento entre presentación, coordinación, generación y persistencia.

- **Separación de responsabilidades:** el controlador actúa como punto de entrada y delega la lógica de procesamiento en servicios y componentes especializados.
- **Procesamiento asíncrono:** Spring Batch desacopla la generación del ciclo principal de la petición HTTP.
- **Persistencia independiente:** la generación del workbook no depende directamente de los detalles de almacenamiento, que se concentran en *ReportService* y *ReportRepository*.
- **Trazabilidad:** Flyway mantiene versionados los cambios del esquema y Spring Batch utiliza sus tablas de metadatos para registrar las ejecuciones.
- **Capacidad de evolución:** la separación por tipo de informe, categoría y servicios comunes facilita futuras ampliaciones sin modificar el punto de entrada principal.

No se realizaron pruebas formales de carga dentro del alcance del proyecto, por lo que no se cuantifica el comportamiento ante niveles elevados de concurrencia. La arquitectura establece, no obstante, una separación que facilita estudiar futuras mejoras de capacidad.

5.2 Diseño de la aplicación

5.2.1 Diseño del cliente

El cliente se divide conceptualmente en dos áreas funcionales: generación e historial. Esta separación evita concentrar toda la interacción en una única pantalla y permite que cada área gestione un flujo claramente diferenciado.

Componente de generación

ReportsGeneratorComponent se encarga de recoger los parámetros del informe y coordinar la interacción con los servicios Angular. Sus responsabilidades de diseño son:

- presentar tipo, categoría, franquicias y rango de fechas;
- cargar dinámicamente las franquicias de cada categoría;
- validar los parámetros antes de enviar la solicitud;
- iniciar la generación;
- mostrar mensajes de estado;
- permitir navegar hacia el historial.

Componente de historial

ReportsHistoryComponent representa la segunda área funcional. Su objetivo es mostrar los informes persistidos de forma paginada y permitir que el usuario recupere un fichero anterior sin volver a generarlo.

La información mostrada incluye los metadatos necesarios para identificar cada informe, mientras que la descarga utiliza el identificador persistente asociado al registro.

Servicios del cliente

La comunicación con el backend se encapsula en servicios Angular. El servicio de descarga y generación inicia la petición, realiza el seguimiento periódico y recupera el fichero cuando está disponible. El servicio de historial consulta las páginas almacenadas y descarga un informe concreto por identificador. Los servicios de franquicias obtienen las opciones disponibles según la categoría seleccionada.

Los flujos asíncronos se modelan mediante Observables de RxJS y las peticiones HTTP se realizan mediante *HttpClient*. Angular Material proporciona los componentes utilizados para formularios, tablas, diálogos y mensajes breves de estado [27, 28].

Validaciones y estados

Antes de enviar una generación se comprueba que exista un tipo y categoría, que se haya seleccionado al menos una franquicia y que el rango de fechas sea coherente cuando se especifica.

Durante la generación se distinguen estados asociados a la validación inicial, inicio del procesamiento, generación en curso, duplicidad, ausencia de datos, disponibilidad del fichero y errores. El seguimiento se realiza mediante *polling*; no se utiliza una conexión persistente como WebSocket o Server-Sent Events.

5.2.2 Diseño del servidor

El backend se organiza en componentes de entrada, coordinación, procesamiento, acceso a datos y utilidades comunes.

Principios de diseño

- **Arquitectura en capas:** separación entre controladores, servicios y repositorios.
- **Inyección de dependencias:** Spring gestiona las dependencias entre los componentes.

- **DTO:** *ReportRequestDto* agrupa los parámetros de la generación y desacopla el contrato de entrada de las entidades de persistencia.
- **Delegación:** el controlador no construye el informe directamente, sino que delega la coordinación en los servicios y la ejecución en Spring Batch.

Organización de los generadores

La lógica se divide en dos ramas principales, correspondientes a informes KPI y Producto. Cada una contempla las categorías Alternativa e Innovación.

Los informes KPI trabajan con instancias, proyectos e indicadores específicos, mientras que los informes de Producto utilizan información destinada a construir elementos como retroplannings, informes diarios y heatmaps. Las operaciones comunes de estilos, celdas y formato se concentran en utilidades reutilizables de Excel.

La selección del generador puede representarse de forma simplificada mediante el siguiente pseudocódigo:

```
1 if type == PRODUCT and category == ALTERNATIVE:
2     generate product alternative report
3 else if type == PRODUCT and category == INNOVATION:
4     generate product innovation report
5 else if type == KPI and category == ALTERNATIVE:
6     generate KPI alternative report
7 else if type == KPI and category == INNOVATION:
8     generate KPI innovation report
9 else:
10    reject invalid configuration
```

Listing 5.1: Selección del generador según los parámetros del informe

5.2.3 Diseño del procesamiento asíncrono

La generación se modela mediante un job de Spring Batch con un único step implementado como *Tasklet*. Esta elección se ajusta al problema porque cada informe constituye una operación completa que combina consulta de datos, cálculos, construcción de múltiples hojas y almacenamiento del workbook.

El flujo diseñado es el siguiente:

1. *ReportBatchController* recibe la solicitud.
2. *ReportLaunchService* coordina la comprobación de duplicidad.
3. Si la solicitud no es duplicada, *JobLauncher* inicia el job de forma asíncrona.

4. *GenerateReportTasklet* recupera los parámetros del job.
5. *ReportDataProbeService* comprueba si existen datos para la combinación solicitada.
6. Si no existen datos, la ejecución finaliza con *NO_DATA*.
7. Si existen datos, se selecciona el generador correspondiente y se construye el workbook.
8. El resultado se entrega a *ReportService*, que centraliza la persistencia.

La separación entre duplicidad y disponibilidad es intencionada: la primera evita lanzar trabajos innecesarios, mientras que la segunda forma parte del propio procesamiento del job.

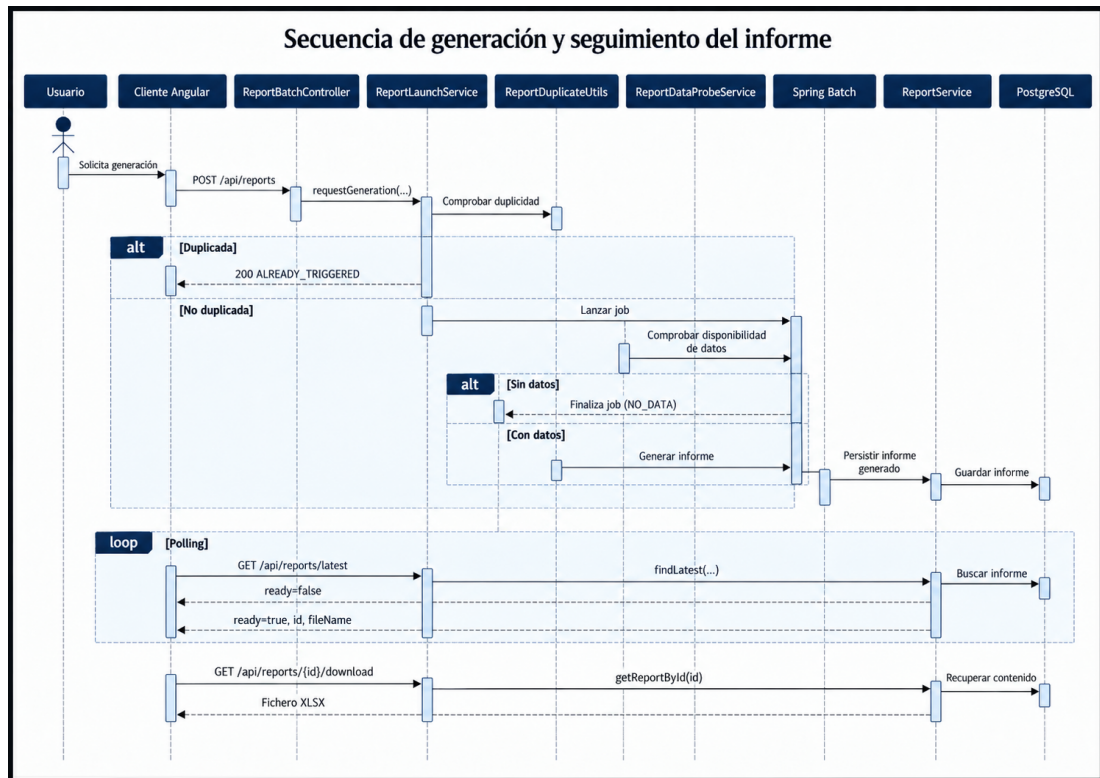


Figura 5.2: Diagrama de secuencia del proceso de generación y seguimiento de un informe, elaborado con Mermaid. Fuente: elaboración propia.

5.2.4 Diseño de la persistencia

El esquema *aesla_reports* contiene la tabla principal *report*, representada en el backend mediante *ReportEntity*. El modelo almacena tanto los metadatos como el contenido binario del workbook.

Tabla 5.2: Atributos de la entidad *Report* en *aesla_reports.report*.

Campo	Tipo lógico (Java / SQL)	Nulo	Descripción
<i>id</i>	UUID / UUID	No	Identificador único del informe (clave primaria).
<i>file_name</i>	String / varchar(512)	No	Nombre del fichero Excel generado.
<i>report_type</i>	enum ReportType / varchar	No	Tipo de informe: <i>KPI</i> o <i>PRODUCT</i> .
<i>report_category</i>	enum ReportCategory / varchar	No	Categoría: <i>INNOVATION</i> o <i>ALTERNATIVE</i> .
<i>created_at</i>	LocalDateTime / timestamp	No	Fecha y hora de creación.
<i>start_date</i>	LocalDate / date	Sí	Fecha inicial del rango solicitado.
<i>end_date</i>	LocalDate / date	Sí	Fecha final del rango solicitado.
<i>franchises</i>	String / text	No	Franquicia o conjunto de franquicias seleccionadas.
<i>file_size</i>	Long / bigint	No	Tamaño del fichero en bytes.
<i>content</i>	byte[] / bytea	No	Contenido binario del workbook.

El uso de un esquema específico permite mantener separados los datos del Report Service. Flyway se utiliza para versionar la creación y evolución de estas estructuras. El detalle de las migraciones implementadas se presenta en el capítulo 6.

5.3 Interacción cliente–servidor

Esta sección concreta los contratos REST utilizados por el diseño.

Tabla 5.3: Operaciones REST principales utilizadas por el Report Service.

Método	Endpoint	Finalidad	Respuesta principal
POST	<i>/api/reports</i>	Solicitar una nueva generación de informe	<i>STARTED</i> , <i>AL-READY_TRIGGERED</i> , <i>NO_DATA</i> o <i>ERROR</i>
GET	<i>/api/reports/latest</i>	Comprobar si existe ya un informe persistido con la configuración solicitada	<i>ready</i> , <i>id</i> , <i>fileName</i>
GET	<i>/api/reports</i>	Consultar informes con filtros y paginación	Página de entidades <i>ReportEntity</i>
GET	<i>/api/reports/{id}/download</i>	Descargar el contenido Excel de un informe almacenado	Fichero XLSX binario
DELETE	<i>/api/reports/{id}</i>	Eliminar un informe por identificador	204, 404 o 500
GET	<i>/alternativeFranchise</i>	Recuperar franquicias de categoría Alternativa	Mapa JSON de franquicias
GET	<i>/innovationFranchise</i>	Recuperar franquicias de categoría Innovación	Mapa JSON de franquicias

5.3.1 Inicio de una generación

El endpoint:

POST /api/reports

recibe los parámetros necesarios para solicitar una nueva generación. El cuerpo de la petición se representa mediante *ReportRequestDto* e incluye el tipo de informe, la categoría, las franquicias y, cuando procede, las fechas inicial y final.

Un ejemplo simplificado es:

```

1 {
2   "reportType": "KPI",
3   "reportCategory": "INNOVATION",
4   "franchises": ["Starbucks", "Vips"],
5   "startDate": "2024-01-01",
6   "endDate": "2024-12-31"
7 }
```

Listing 5.2: Ejemplo de solicitud de generación en JSON

Antes de iniciar el procesamiento, *ReportBatchController* transforma los valores de tipo y categoría a los enumerados utilizados por el backend y delega en *ReportParamService* la normalización de fechas y franquicias. Posteriormente, la solicitud se entrega a *ReportLaunchService.requestGeneration()*.

La respuesta del endpoint está formada por los campos *code* y *message*. Tanto el código HTTP como el valor lógico devuelto dependen del resultado de la operación de lanzamiento:

- **202 Accepted – STARTED**: la solicitud ha sido aceptada y la generación ha comenzado correctamente.
- **200 OK – ALREADY_TRIGGERED**: ya existe una generación con la misma configuración dentro de la ventana temporal de cinco minutos, por lo que no se inicia un nuevo procesamiento.
- **200 OK – NO_DATA**: no existen datos disponibles para construir el informe solicitado.
- **500 Internal Server Error – ERROR**: se ha producido un error durante el intento de iniciar la generación.

Por ejemplo, una generación iniciada correctamente devuelve:

```
1 {  
2   "code": "STARTED",  
3   "message": "Generacion del informe iniciada."  
4 }
```

Listing 5.3: Ejemplo de respuesta al iniciar una generación

Cuando la respuesta es *STARTED*, el frontend inicia el mecanismo de seguimiento mediante consultas periódicas a *GET /api/reports/latest*.

5.3.2 Seguimiento de la generación

El seguimiento de una generación se realiza mediante el endpoint:

GET /api/reports/latest

La petición incluye como parámetros el tipo de informe, la categoría, las fechas opcionales y las franquicias asociadas a la generación. Antes de realizar la consulta, *ReportParamService* normaliza estos valores de la misma forma que durante el inicio de la generación.

A diferencia de una API de monitorización del estado interno de Spring Batch, este endpoint no devuelve directamente el estado del job. Su función consiste en comprobar mediante

ReportService.findLatest() si ya existe un informe persistente que coincida con la configuración solicitada.

La respuesta utiliza la estructura *LatestReportResponse*, formada por tres atributos:

- *ready*: indica si se ha encontrado un informe disponible;
- *id*: identificador UUID del informe cuando este ya existe;
- *fileName*: nombre del fichero almacenado.

Mientras el informe todavía no se encuentra persistente, el endpoint devuelve:

```
1 {  
2   "ready": false,  
3   "id": null,  
4   "fileName": null  
5 }
```

Listing 5.4: Respuesta de seguimiento mientras el informe no está disponible

Cuando la generación ha finalizado y el informe correspondiente ya se encuentra almacenado, la respuesta adopta la siguiente forma:

```
1 {  
2   "ready": true,  
3   "id": "550e8400-e29b-41d4-a716-446655440000",  
4   "fileName": "KPI_Innovation_2026-09-05.xlsx"  
5 }
```

Listing 5.5: Respuesta de seguimiento cuando el informe está disponible

El frontend utiliza esta operación mediante un mecanismo de *polling*. Por tanto, un valor *ready=false* no debe interpretarse como un estado concreto de Spring Batch, sino únicamente como la ausencia, en ese momento, de un informe persistente que coincida con los parámetros consultados.

Los estados *NO_DATA*, *ALREADY_TRIGGERED* y *ERROR* se gestionan durante la petición inicial *POST /api/reports* y no forman parte de *LatestReportResponse*.

5.3.3 Consulta del historial

La consulta de los informes almacenados se realiza mediante el endpoint:

GET /api/reports

La operación devuelve una página de entidades *ReportEntity* y utiliza el mecanismo *Pagable* de Spring Data para gestionar la paginación. Por ejemplo:

GET /api/reports?page=0&size=5

Además de la paginación, el backend permite aplicar diferentes filtros opcionales sobre los informes almacenados:

- *type*: tipo de informe, como KPI o Producto;
- *category*: categoría Innovación o Alternativa;
- *createdStart* y *createdEnd*: intervalo correspondiente a la fecha y hora de creación del informe;
- *fileName*: filtrado por nombre del fichero;
- *startDate* y *endDate*: fechas asociadas a los parámetros utilizados durante la generación;
- *franchise*: franquicia asociada al informe;
- parámetros de *Pageable*, como *page*, *size* y los criterios de ordenación admitidos por Spring Data.

Los parámetros no enviados permanecen a *null*, por lo que *ReportService.getReportsWithFilters()* puede construir la consulta teniendo en cuenta únicamente los criterios solicitados.

Un ejemplo de consulta filtrada podría ser:

GET /api/reports?type=KPI&category=INNOVATION&page=0&size=5

Una respuesta simplificada presenta la estructura paginada siguiente:

```
1 {
2   "content": [
3     {
4       "id": "uuid",
5       "fileName": "KPI_Innovation_2024-11-15.xlsx",
6       "reportType": "KPI",
7       "reportCategory": "INNOVATION",
8       "createdAt": "2024-11-15T10:30:00",
9       "franchises": "Starbucks,Vips",
10      "fileSize": 1048576
11    }
12  ],
13  "totalElements": 42,
14  "totalPages": 9,
15  "number": 0
16 }
```

Listing 5.6: Ejemplo de respuesta paginada del historial en JSON

Aunque el backend dispone de estos criterios de filtrado, la interfaz desarrollada durante el alcance del proyecto no tiene por qué exponer necesariamente todos ellos como controles visibles. Su disponibilidad en la API permite, no obstante, ampliar posteriormente las capacidades de búsqueda del historial sin tener que rediseñar el contrato principal del backend.

5.3.4 Descarga de un informe

La descarga de un informe almacenado se realiza mediante:

GET /api/reports/{id}/download

El identificador UUID permite localizar de forma unívoca el informe persistido. El controlador delega la recuperación en *ReportService.getReportById()*, obtiene el contenido almacenado y lo devuelve como un fichero Excel.

La respuesta incorpora la cabecera *Content-Disposition* con el nombre original del fichero y utiliza el tipo MIME correspondiente al formato XLSX:

application/vnd.openxmlformats-officedocument.spreadsheetml.sheet

De este modo, la descarga de un informe desde el historial no vuelve a ejecutar la lógica de generación, sino que recupera directamente el contenido binario previamente almacenado en PostgreSQL.

La figura 5.3 representa la secuencia completa de consulta y redescarga desde el historial, mostrando la interacción entre el componente Angular, el servicio del frontend, el controlador REST y la capa de persistencia.

Secuencia de consulta y redescarga desde el historial

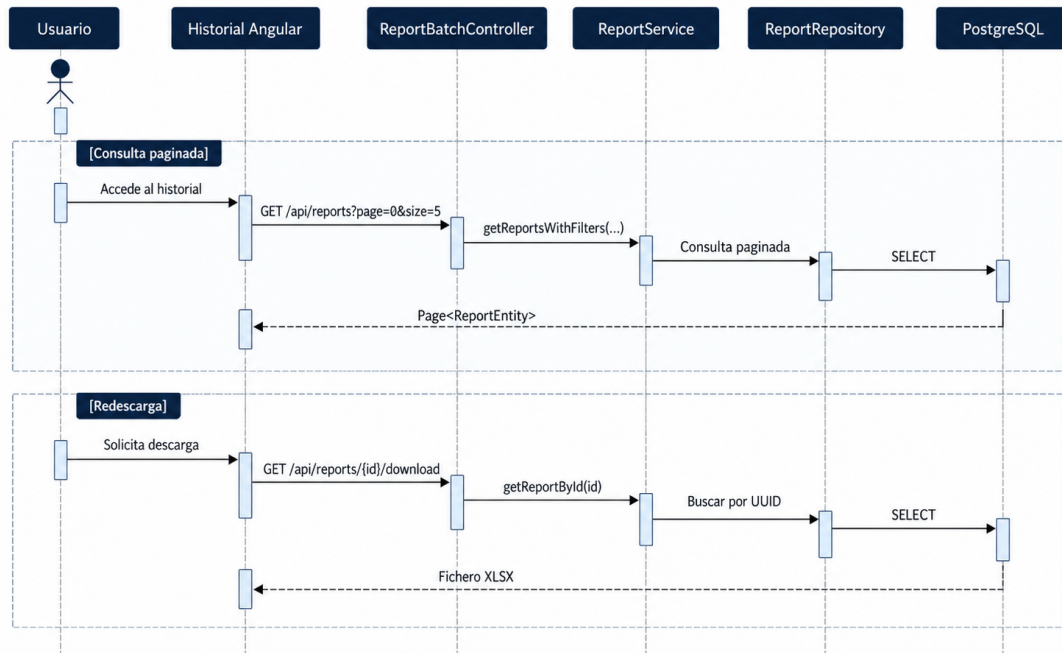


Figura 5.3: Diagrama de secuencia de la consulta y redescarga de informes desde el historial, elaborado con Mermaid. Fuente: elaboración propia.

5.3.5 Eliminación de un informe

El backend dispone además de una operación para eliminar un informe persistente:

DELETE /api/reports/{id}

La operación utiliza el UUID del informe y delega la eliminación en *ReportService.deleteReportById()*. El código HTTP devuelto depende del resultado:

- *204 No Content* cuando el informe existe y se elimina correctamente;
- *404 Not Found* cuando no existe ningún informe asociado al identificador solicitado;
- *500 Internal Server Error* si se produce una excepción durante la operación.

Esta capacidad está disponible en el backend, aunque la versión final de la interfaz desarrollada dentro del alcance del proyecto no incorporó todavía los controles necesarios para ejecutar la eliminación desde la página de historial.

5.3.6 Manejo de errores y respuestas HTTP

Los controladores utilizan códigos HTTP y cuerpos de respuesta diferentes según el tipo de operación.

En la solicitud de generación, una ejecución iniciada correctamente devuelve *202 Accepted*, mientras que las situaciones de duplicidad y ausencia de datos se representan mediante *200 OK* acompañadas de los códigos lógicos *ALREADY_TRIGGERED* y *NO_DATA*, respectivamente. Si se produce una excepción durante el inicio de la generación, se devuelve *500 Internal Server Error* con el código *ERROR*.

En la operación de eliminación, el backend diferencia entre *204 No Content*, *404 Not Found* y *500 Internal Server Error*.

Por su parte, *GET /api/reports/latest* devuelve normalmente *200 OK*; la existencia o ausencia del informe no se representa mediante el código HTTP, sino mediante el atributo booleano *ready* de *LatestReportResponse*.

Esta separación permite utilizar el protocolo HTTP para expresar el resultado general de la operación y, cuando resulta necesario, añadir códigos de aplicación específicos que el frontend puede transformar en mensajes comprensibles para el usuario.

Implementación y validación

Este capítulo describe cómo se materializó el diseño presentado en el capítulo 5. Se recogen las modificaciones realizadas en el backend y el frontend, la incorporación de persistencia y procesamiento asíncrono, las tareas de calidad y mantenimiento y, finalmente, la estrategia de validación aplicada antes de integrar los cambios.

6.1 Organización de la implementación

6.1.1 Organización de los work items

El trabajo se organizó en Jira mediante el EPIC *Improvements for AESLA Project in 2025 Q3 & Q4*. Este elemento actuó como padre común de los work items relacionados con la evolución del Report Service. La mayoría fueron tareas técnicas, aunque también se registraron incidencias de tipo *Bug* y una historia de usuario de prioridad alta relacionada con la introducción del procesamiento asíncrono mediante Spring Batch.

Cada work item se desarrolló en una rama independiente identificada con su número. En ella se realizaron los commits y, al finalizar, se abrió un Pull Request hacia [development](#). La integración se producía después de superar la revisión de código, las comprobaciones automáticas y la validación en QA. Los elementos asociados a la entrega compartían la misma [fix version](#), facilitando la trazabilidad de los cambios incorporados.

Entre los trabajos realizados se incluyeron la persistencia y consulta de informes, la generación asíncrona, el control de duplicidad, la incorporación de logs, la mejora de las páginas de generación e historial, la reducción de duplicación, el aumento de cobertura, la gestión de vulnerabilidades y la resolución de problemas de inicialización de Spring Batch.

6.2 Implementación del backend

6.2.1 Estructura física del proyecto

El backend está ubicado en *main/java/com/gbtec/bic/cloud*. Los paquetes más relevantes para el Report Service son:

- **config**: configuración general de Spring, incluyendo Aspose Cells, seguridad y CORS.
- **datastorage.elasticsearch**: integración con Elasticsearch y consulta de variables de procesos.
- **processes.batch**: configuración de Spring Batch, controlador REST, DTO de entrada, preparación de parámetros, tasklet y utilidades relacionadas con duplicidad y nombres de fichero.
- **kpi_reports**: lógica específica de los informes KPI para las categorías Alternativa e Innovación.
- **product_reports**: lógica específica de los informes de Producto para las categorías Alternativa e Innovación.
- **report_probe**: servicios de comprobación de disponibilidad y coordinación del lanzamiento.
- **report_storage**: entidad, repositorio y servicio de persistencia de informes.
- **utils**: utilidades compartidas de construcción y formato de Excel.

Esta organización refleja en el código la separación de responsabilidades descrita en el diseño.

6.3 Unificación de la gestión de informes

Una de las tareas de mayor alcance realizadas durante la evolución del backend consistió en unificar el tratamiento de las solicitudes de generación. La implementación original había evolucionado de forma independiente para los distintos tipos y categorías de informe, lo que provocaba diferencias en el tratamiento de parámetros y aumentaba la cantidad de lógica repetida.

El objetivo de esta refactorización no fue eliminar la especialización de los generadores, ya que los informes KPI y Producto continúan trabajando con fuentes de datos y estructuras diferentes, sino establecer un flujo común para aquellas operaciones que son independientes del contenido concreto del workbook.

6.3.1 Unificación del contrato de entrada

Las solicitudes de generación se centralizaron mediante *ReportRequestDto*, que agrupa los principales parámetros de negocio:

- tipo de informe;
- categoría;
- franquicias;
- fecha inicial opcional;
- fecha final opcional.

De este modo, *ReportBatchController* dispone de un único punto de entrada para iniciar las generaciones mediante *POST /api/reports*, independientemente de si se solicita un informe KPI o Producto y de si su categoría es Innovación o Alternativa.

6.3.2 Normalización de parámetros

La unificación requirió también homogeneizar el tratamiento de valores opcionales y representaciones diferentes de un mismo parámetro. *ReportParamService* se utiliza en la frontera de la API para transformar los datos recibidos en una representación común antes de entregarlos a los siguientes componentes.

Este paso resulta especialmente relevante para las fechas opcionales y para las franquicias, ya que los mismos valores normalizados se reutilizan tanto al iniciar una generación como al buscar posteriormente el informe mediante */api/reports/latest*.

Dentro de la lógica de generación se utilizan además las operaciones comunes de *ReportParamUtils* para evitar repetir transformaciones de parámetros en los diferentes generadores y servicios de comprobación.

6.3.3 Separación entre flujo común y lógica específica

Tras la unificación, las responsabilidades comunes quedan concentradas en los componentes de coordinación:

- *ReportBatchController* recibe la solicitud;
- *ReportParamService* prepara los parámetros;

- *ReportLaunchService* coordina el lanzamiento;
- *ReportDuplicateUtils* evita ejecuciones equivalentes recientes;
- *Spring Batch* gestiona la ejecución asíncrona;
- *ReportDataProbeService* gestiona la disponibilidad de los datos;
- *ReportService* centraliza la persistencia y recuperación.

La lógica específica se mantiene separada en los paquetes de informes KPI y Producto y, dentro de ellos, en las categorías Innovación y Alternativa. Esta estructura permite reutilizar el flujo común sin eliminar las diferencias de negocio necesarias para construir cada workbook.

La unificación redujo la necesidad de mantener caminos paralelos para operaciones equivalentes y simplificó la incorporación posterior de persistencia, control de duplicidad y seguimiento desde el frontend.

6.4 Implementación de la persistencia y el historial

6.4.1 Integración de Flyway

Una parte relevante del trabajo fue incorporar un mecanismo persistente para los informes y gestionar de forma versionada los cambios de base de datos. Flyway se integró en el backend para aplicar las migraciones en orden y registrar su ejecución en *flyway_schema_history* [18, 29].

La integración comenzó con la dependencia correspondiente en *pom.xml*:

```

1 <dependency>
2   <groupId>org.flywaydb</groupId>
3   <artifactId>flyway-core</artifactId>
4 </dependency>
```

Listing 6.1: Dependencia de Flyway añadida al backend

A continuación se creó la ubicación de migraciones dentro de *src/main/resources/db/migration*:

```

1 src/main/resources/
2   db/
3     migration/
4       V1.0.0.1__initial.sql
5       V1.0.0.2__create_report_table.sql
6       V1.0.0.3__create_spring_batch_tables.sql
```

Listing 6.2: Estructura de migraciones de Flyway

Esta estructura permite que las migraciones formen parte del propio código versionado del servicio y puedan aplicarse de manera consistente en los distintos entornos.

6.4.2 Migraciones implementadas

Tabla 6.1: Migraciones de Flyway incorporadas durante el proyecto.

Migración	Objetivo	Elementos principales
V1.0.0.1	Crear el esquema del Report Service	Esquema <i>aesla_reports</i> .
V1.0.0.2	Incorporar la persistencia de informes	Tabla <i>aesla_reports.report</i> con contenido binario y metadatos.
V1.0.0.3	Crear la infraestructura de metadatos de Spring Batch	Seis tablas de metadatos, tres secuencias e índices auxiliares.

V1.0.0.1: creación del esquema

La primera migración, *V1.0.0.1__initial.sql*, crea el esquema específico:

```
1 CREATE SCHEMA IF NOT EXISTS aesla_reports;
```

Listing 6.3: Creación del esquema del Report Service

Separar los datos en *aesla_reports* permite aislar la persistencia propia del Report Service respecto a otras estructuras utilizadas por la plataforma.

V1.0.0.2: creación de la tabla *report*

La segunda migración, *V1.0.0.2__create_report_table.sql*, crea *aesla_reports.report*. La tabla almacena el contenido binario del workbook y los metadatos descritos en el cuadro 5.2: identificador, nombre de fichero, tipo, categoría, fecha de creación, fechas opcionales, franquicias, tamaño y contenido.

Sobre esta estructura se implementó *ReportEntity*. El acceso se encapsuló mediante *ReportRepository*, basado en *JpaRepository*, y la lógica de almacenamiento y recuperación se centralizó en *ReportService*.

Esta separación evita que *GenerateReportTasklet* tenga que conocer los detalles de serialización y persistencia. El tasklet entrega el workbook al servicio y este se encarga de convertirlo a bytes, construir la entidad y guardarla mediante el repositorio.

V1.0.0.3: tablas de Spring Batch

La tercera migración, *V1.0.0.3__create_spring_batch_tables.sql*, incorpora las estructuras necesarias para el repositorio de metadatos de Spring Batch. La definición se basó en el esquema de referencia proporcionado por la documentación oficial [30].

Se crearon las tablas:

- *BATCH_JOB_INSTANCE*;
- *BATCH_JOB_EXECUTION*;
- *BATCH_JOB_EXECUTION_PARAMS*;
- *BATCH_STEP_EXECUTION*;
- *BATCH_STEP_EXECUTION_CONTEXT*;
- *BATCH_JOB_EXECUTION_CONTEXT*.

También se añadieron las secuencias:

- *BATCH_STEP_EXECUTION_SEQ*;
- *BATCH_JOB_EXECUTION_SEQ*;
- *BATCH_JOB_INSTANCE_SEQ*.

Además de las tablas y secuencias, se añadieron índices sobre algunas de las claves utilizadas para relacionar los metadatos de ejecución:

- *BATCH_JOB_EXECUTION_PARAMS*(*JOB_EXECUTION_ID*);
- *BATCH_STEP_EXECUTION*(*JOB_EXECUTION_ID*);
- *BATCH_JOB_EXECUTION*(*JOB_INSTANCE_ID*).

Por último, se incorporaron índices sobre las claves utilizadas con mayor frecuencia en las relaciones entre ejecuciones, steps e instancias de job.

Como resultado, la aplicación pasó a disponer de una estructura versionada tanto para los informes como para la información de ejecución necesaria por Spring Batch.

6.4.3 Consulta y recuperación de informes

La incorporación de persistencia permitió implementar el historial. La información almacenada incluye la fecha y hora de generación, el nombre y tamaño del fichero, el contenido y los parámetros necesarios para identificar la configuración del informe.

ReportRepository proporciona las operaciones de acceso a PostgreSQL, mientras que *ReportService* concentra la recuperación del registro y del contenido binario. Esta funcionalidad permite consultar generaciones anteriores y redescargarlas sin ejecutar nuevamente la lógica de negocio que construye el workbook.

6.5 Implementación de la generación asíncrona

6.5.1 Configuración de Spring Batch

La historia de usuario de mayor alcance introdujo la generación en segundo plano. Para ello se configuraron *BatchAsyncConfiguration* y *ReportBatchConfig*. La primera define el executor asíncrono y la segunda configura el job y el step de generación.

El job utiliza un único step implementado mediante *GenerateReportTasklet*. Esta estructura permitió conservar la lógica existente de generación de Excel y envolverla dentro de un proceso gestionado por Spring Batch sin transformarla artificialmente al modelo *ItemReader–ItemProcessor–ItemWriter*.

6.5.2 Control de duplicidad mediante metadatos de Spring Batch

Para evitar que una misma solicitud desencadene varias generaciones en un intervalo corto de tiempo se implementó *ReportDuplicateUtils*. A diferencia de la persistencia de informes, esta comprobación no se realiza consultando la tabla *aesla_reports.report*, sino utilizando los metadatos de ejecución de Spring Batch mediante *JobExplorer*.

La utilidad define una ventana temporal de cinco minutos. Para determinar si una solicitud ya ha sido desencadenada recientemente, calcula el instante límite restando esta ventana a la hora actual y analiza las ejecuciones asociadas al job de generación.

En primer lugar se consultan mediante *findRunningJobExecutions()* las ejecuciones que permanecen activas. Posteriormente se revisan también las instancias recientes recuperadas por *getJobInstances()*, analizando sus correspondientes ejecuciones. En ambos casos solo se

consideran aquellas cuyo *createTime* es posterior al límite temporal calculado.

Dos ejecuciones se consideran equivalentes cuando coinciden los parámetros de negocio almacenados en sus *JobParameters*:

- *reportType*;
- *reportCategory*;
- *franchises*;
- *startDate*;
- *endDate*.

La comparación se realiza mediante igualdad exacta sobre los valores ya preparados por el flujo de parámetros. Para las fechas opcionales se compara su representación textual cuando existe un valor y *null* cuando no se ha indicado.

De forma simplificada, el algoritmo puede representarse así:

```
1 threshold = currentTime - 5 minutes
2
3 for execution in runningExecutions:
4     if execution.createTime > threshold
5         and sameBusinessParameters(execution):
6         return true
7
8 for instance in recentJobInstances:
9     for execution in executions(instance):
10        if execution.createTime > threshold
11            and sameBusinessParameters(execution):
12            return true
13
14 return false
```

Listing 6.4: Pseudocódigo del control de duplicidad

Este enfoque permite detectar tanto una generación que todavía está en curso como otra que haya finalizado recientemente. Además, al basarse en los metadatos mantenidos por Spring Batch, la comprobación queda asociada a las ejecuciones reales del job y no exclusivamente a la existencia final de un fichero en la tabla de informes.

6.5.3 Comprobación de disponibilidad de datos

Una vez preparados los parámetros de la generación, el sistema debe comprobar si existe información suficiente para construir el informe solicitado. Esta responsabilidad se concentra

en *ReportDataProbeService*, evitando ejecutar toda la lógica de creación de un workbook cuando las fuentes no contienen datos relevantes.

La implementación selecciona la estrategia de comprobación en función de la combinación formada por el tipo y la categoría del informe.

Informes de Producto

Para los informes de Producto se utilizan los servicios de exportación correspondientes a cada categoría:

- *AlternativeExportDataService* para la categoría Alternativa;
- *InnovationExportDataService* para la categoría Innovación.

Antes de realizar las consultas se normalizan los parámetros necesarios, incluidas las franquicias y las fechas opcionales, mediante las utilidades comunes de *ReportParamUtils*. La comprobación determina si los procesos obtenidos contienen tareas relevantes para la construcción del informe.

Informes KPI

En los informes KPI se utilizan los servicios especializados en la obtención de instancias y proyectos:

- *KpiAlternativeInstanceService* para Alternativa;
- *KpiInnovationInstanceService* para Innovación.

En este caso, la comprobación se basa en determinar si existen proyectos asociados a las franquicias solicitadas que puedan utilizarse en los cálculos del informe.

De forma resumida:

```

1 if reportType == PRODUCT:
2     normalize parameters
3     if category == ALTERNATIVE:
4         data = AlternativeExportDataService
5     else:
6         data = InnovationExportDataService
7     return data contains relevant tasks
8
9 if reportType == KPI:
10     normalize franchises
11     if category == ALTERNATIVE:

```

```
12     data = KpiAlternativeInstanceService
13     else:
14         data = KpiInnovationInstanceService
15     return data contains relevant projects
```

Listing 6.5: Selección de la comprobación de disponibilidad de datos

Esta separación mantiene centralizada la decisión de si una generación tiene datos disponibles, mientras que la recuperación concreta continúa delegada en los servicios especializados ya existentes para cada tipo de informe.

6.5.4 Generación y almacenamiento

Los generadores KPI y Producto recuperan la información desde BIC Process Execution y Elasticsearch, realizan las transformaciones necesarias y generan las distintas hojas con Aspose Cells.

La refactorización realizada durante el proyecto permitió concentrar operaciones compartidas de formato y creación de celdas en utilidades comunes. Una vez finalizada la construcción, *GenerateReportTasklet* delega el almacenamiento en *ReportService*.

6.6 Implementación del frontend

6.6.1 Estructura de módulos y componentes

El frontend está ubicado en *webapp/custom-aesla-angular-front*. Dentro de *src/app*, los elementos más relevantes son:

- **reports-generator:**
 - *reports-generator.component.ts*;
 - *reports-generator.component.html*;
 - *reports-generator.component.scss*;
 - *reports-generator.component.spec.ts*.
- **reports-history:**
 - *reports-history.component.ts*;
 - *reports-history.component.html*;
 - *reports-history.component.scss*;
 - *reports-history.component.spec.ts*;

- *dialog/*.
- **services:**
 - *get-alternative-franchise-list.service.ts*;
 - *get-innovation-franchise-list.service.ts*;
 - *report-download.service.ts*;
 - *reports-history.service.ts*.
- **Archivos raíz:** *app.component.ts*, *app.module.ts* y *app-routing.module.ts*.

6.6.2 Generación y seguimiento mediante polling

ReportDownloadService se adaptó para desacoplar la descarga del inicio de la generación. Tras enviar *POST /api/reports*, el cliente comienza a consultar */api/reports/latest* cada cinco segundos.

El seguimiento se mantiene durante un máximo de 120 segundos. Cuando el backend indica *ready=true* y devuelve el identificador del informe, el cliente solicita */api/reports/{id}/download* y recibe el contenido como *blob*. Si se supera el tiempo máximo, se informa al usuario de que el informe puede consultarse posteriormente desde Historial.

También se contemplan mensajes específicos para duplicidad, ausencia de datos, errores de validación y errores de red.

6.6.3 Página de historial

Se implementó *ReportsHistoryComponent* junto con *ReportsHistoryService*. El servicio recupera páginas de informes, transforma las respuestas al modelo TypeScript y solicita el contenido binario cuando el usuario selecciona una descarga.

La interfaz presenta los registros de más reciente a más antiguo y utiliza paginación. De esta forma, la redescarga queda desacoplada de la generación original.

El backend dispone también de la operación *DELETE /api/reports/{id}*, que permite eliminar un informe persistido a partir de su identificador. No obstante, la integración de esta funcionalidad en el frontend no quedó incluida en la versión final desarrollada dentro del alcance del proyecto.

En consecuencia, la página de historial permite actualmente consultar y redescargar informes, mientras que la incorporación de controles de eliminación, un diálogo de confirmación y la gestión visual del resultado se mantiene como una ampliación posterior.

6.7 Mejoras de calidad y mantenimiento

6.7.1 Gestión de vulnerabilidades

El frontend presentaba inicialmente vulnerabilidades asociadas a sus dependencias. Parte del problema estaba relacionado con la coexistencia de npm y Yarn, que podía producir resoluciones diferentes del árbol de dependencias.

La estrategia aplicada tuvo dos pasos:

1. **Estandarización en Yarn:** se eliminaron los ficheros de bloqueo del gestor que dejó de utilizarse, se configuró Yarn como gestor único y se realizó un proceso de deduplicación y reinstalación.
2. **Mitigación mediante *resolutions*:** dado que una migración completa de Angular quedaba fuera del alcance, se forzaron versiones parcheadas de determinadas dependencias transitivas cuando era posible.

Tras estas tareas permanecieron tres CVE menores vinculadas al *tooling* de Angular 15. La actualización del framework se mantiene como línea de trabajo futuro.

6.7.2 Refactorización y reducción de duplicación

La refactorización se centró en extraer operaciones comunes, homogeneizar el tratamiento de parámetros y reducir lógica repetida entre tipos y categorías de informe.

Entre las acciones realizadas se encuentran:

- extracción de métodos y utilidades compartidas;
- normalización común de franquicias y fechas;
- centralización de operaciones de estilo y creación de celdas Excel;
- parametrización de lógica repetida;
- separación de responsabilidades entre lanzamiento, comprobación de datos, generación y persistencia.

6.7.3 Observabilidad y soporte

Se añadieron logs informativos en puntos relevantes del proceso de exportación y generación. Estos registros permiten seguir el avance de una ejecución y localizar con mayor rapidez posibles incidencias.

También se revisaron bloques de tratamiento de excepciones en servicios concretos para registrar información útil antes de propagar o gestionar el error correspondiente.

6.7.4 Evolución de métricas de calidad

Al inicio del proyecto, la cobertura medida por SonarCloud era del 17,7%, por debajo del umbral interno del 20%. Se añadieron pruebas unitarias con JUnit 5 y Mockito y pruebas de integración con Testcontainers sobre los componentes relacionados con persistencia y migraciones.

La cobertura final alcanzó el 25,7%. En paralelo, la duplicación descendió del 15,8% al 10,0%.

Tabla 6.2: Evolución de las principales métricas de calidad medidas con SonarCloud.

Métrica	Valor inicial	Valor final	Variación
Cobertura de código	17,7%	25,7%	+8,0 puntos porcentuales
Duplicación de código	15,8%	10,0%	-5,8 puntos porcentuales

Aunque el valor final de cobertura continúa reflejando la existencia de código legado no cubierto por pruebas automatizadas, el alcance del proyecto se centró principalmente en incrementar la cobertura de los componentes modificados y superar el umbral mínimo del 20% establecido para la integración de cambios. Por tanto, el incremento obtenido debe interpretarse en el contexto de una aplicación preexistente cuyo código completo no formaba parte del alcance de este trabajo.

Las capturas completas de SonarCloud utilizadas como evidencia de estas métricas se incluyen en el apéndice [A.2](#).

6.8 Incidencias y trabajo no realizado

Durante la integración de Spring Batch se detectó una incidencia relacionada con la inicialización de su esquema de metadatos. La configuración inicial no creaba correctamente las estructuras mediante Flyway antes del lanzamiento de los jobs. El problema se resolvió ajustando la configuración y las migraciones para garantizar que las tablas estuvieran

disponibles durante el arranque.

También se planteó una historia de usuario para permitir la eliminación de informes desde el historial. Esta funcionalidad no llegó a implementarse dentro del alcance del proyecto y se mantiene como posible ampliación futura.

6.9 Validación y pruebas

La estrategia de validación se organizó en tres niveles complementarios: pruebas unitarias sobre la lógica de aplicación y las utilidades, pruebas de integración sobre persistencia y migraciones, y pruebas manuales funcionales sobre los flujos completos en QA.

6.9.1 Relación entre requisitos y pruebas

La tabla 6.3 relaciona los requisitos funcionales del capítulo 4 con los mecanismos principales utilizados para validarlos.

Tabla 6.3: Trazabilidad entre requisitos funcionales y pruebas realizadas.

Requisito	Comportamiento validado	Tipo de prueba	Casos
RF-01	Generación asíncrona, validación de parámetros, ausencia de datos y seguimiento	Unitarias, integración y QA	CP-01, CP-02, CP-06, CP-07, CP-08
RF-02	Detección de solicitudes equivalentes dentro de la ventana temporal	Unitarias y QA	CP-03
RF-03	Persistencia, descarga y redescarga posterior	Integración y QA	CP-05, CP-08
RF-04	Consulta, ordenación y paginación del historial	Integración y QA	CP-04

6.9.2 Catálogo representativo de pruebas

Tabla 6.4: Casos de prueba representativos del Report Service.

ID	Caso de prueba	Resultado esperado	Resultado
CP-01	Generar un informe KPI de Innovación con datos disponibles	La solicitud devuelve <i>STARTED</i> , se procesa el informe y este queda almacenado	OK
CP-02	Solicitar una generación para la que no existen datos	La petición devuelve el código <i>NO_DATA</i> y no se genera un informe descargable	OK
CP-03	Repetir una solicitud equivalente dentro de la ventana de cinco minutos	Se devuelve <i>ALREADY_TRIGGERED</i> y no se inicia una segunda generación	OK
CP-04	Consultar el listado paginado de informes	Los informes se recuperan de forma paginada y conforme a los filtros solicitados	OK
CP-05	Descargar un informe almacenado	Se recupera correctamente el fichero Excel asociado al identificador	OK
CP-06	Introducir un rango de fechas inválido en la interfaz	El frontend impide el envío de la solicitud y muestra el mensaje de validación correspondiente	OK
CP-07	Consultar <code>/api/reports/latest</code> mientras el informe aún no se ha persistido	Se devuelve <i>ready=false</i> con identificador y nombre de fichero nulos	OK
CP-08	Consultar mediante <i>polling</i> un informe ya finalizado	Se devuelve <i>ready=true</i> junto con el identificador y el nombre del fichero, permitiendo iniciar la descarga	OK

Estos casos constituyen una selección representativa de los escenarios funcionales y de integración más relevantes.

6.9.3 Pruebas manuales y entorno de QA

Para cada work item se siguió un flujo de validación en un entorno controlado antes de integrar los cambios en las ramas principales. El proceso utilizaba la infraestructura de Ansible gestionada mediante AWX y el registro de imágenes Quay.

1. **Desarrollo y commits:** implementación del cambio en su rama.

2. **Actualización de versión:** modificación del número de versión cuando correspondía para mantener la trazabilidad.
3. **Build y publicación:** GitHub Actions construía la imagen Docker y la publicaba en Quay.
4. **Despliegue:** AWX ejecutaba el playbook necesario para desplegar la versión en QA.
5. **Validación funcional:** se comprobaban generación, ausencia de datos, duplicidad, historial, descarga, mensajes de error y *polling*.
6. **Iteración o cierre:** si aparecía una incidencia se corregía y repetía el flujo. Cuando el resultado era correcto, la tarea se marcaba como *QA OK* y continuaba hacia la integración.

La combinación de pruebas automatizadas y validación funcional permitió comprobar los principales flujos antes de su integración en *development*.

Solución desarrollada

7.1 Resultado funcional global

La solución final integra en un único flujo las mejoras diseñadas e implementadas durante el proyecto: persistencia de informes, generación asíncrona, seguimiento desde el frontend, control de duplicidad, historial, redescarga y mejoras en la información mostrada al usuario.

A diferencia de la versión inicial, el Report Service ya no limita su función a generar un fichero y devolverlo inmediatamente. El informe pasa a formar parte de un ciclo de vida que comprende la solicitud, el procesamiento, el almacenamiento, la consulta y la recuperación posterior.

La tabla [7.1](#) resume las principales diferencias entre la situación inicial y la solución desarrollada.

Tabla 7.1: Comparación entre la versión inicial y la solución desarrollada.

Aspecto	Situación inicial	Solución desarrollada
Generación	Procesamiento síncrono	Procesamiento asíncrono gestionado mediante Spring Batch
Persistencia	Sin almacenamiento histórico	Informes y metadatos almacenados en PostgreSQL
Historial	No disponible	Consulta paginada de informes almacenados
Duplicidad	Sin control específico	Comprobación de ejecuciones equivalentes recientes mediante metadatos de Spring Batch
Seguimiento	Información limitada durante la generación	Seguimiento mediante <i>polling</i> hasta detectar la persistencia del informe
Recuperación	Dependiente de una nueva generación	Descarga por identificador sin necesidad de regenerar el informe

7.2 Página de Generar

La página de generación constituye el punto de entrada principal de la aplicación. Permite configurar los parámetros necesarios para crear un informe y proporciona información inmediata sobre las validaciones, el inicio del procesamiento y el resultado del seguimiento.

Durante una primera fase se valoró incluir una pantalla inicial que actuase como menú entre generación e historial. Tras revisar el flujo con el equipo se consideró que añadía una navegación innecesaria, por lo que la solución final mantiene dos interfaces principales: *Generar informe* e *Historial*, utilizando la primera como página de entrada.

7.2.1 Funcionalidad

- configuración de tipo, categoría, franquicias y fechas;
- carga dinámica de franquicias;
- validación de los parámetros antes de enviar la solicitud;
- inicio de la generación asíncrona;
- seguimiento periódico del resultado;

- descarga cuando el informe está disponible;
- acceso directo a la página de historial.

7.2.2 Comportamiento de la interfaz

Los controles relevantes se deshabilitan cuando existe una operación en curso y los mensajes permiten distinguir validaciones incorrectas, solicitudes duplicadas, generación iniciada, ausencia de datos, errores y disponibilidad del fichero.

Cuando el backend detecta una solicitud equivalente reciente, la interfaz evita iniciar una nueva generación e informa al usuario de que puede consultar el informe desde Historial.

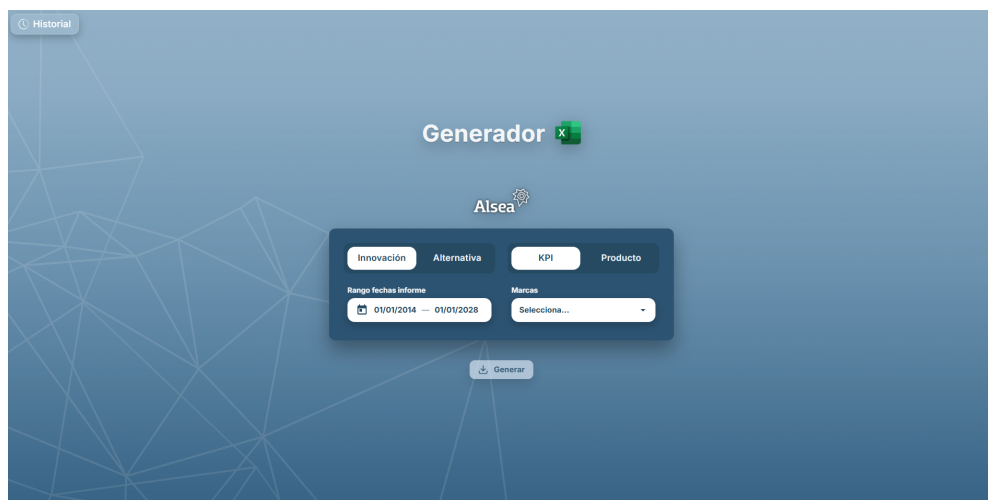


Figura 7.1: Interfaz principal del módulo de generación de informes. Fuente: captura propia.

Tabla 7.2: Principales estados gestionados por la interfaz durante la generación.

Situación	Respuesta o estado	Comportamiento de la interfaz
Parámetros inválidos	Validación local	No se envía la petición y se muestra el error correspondiente.
Generación aceptada	<i>STARTED</i>	Se informa del inicio y comienza el mecanismo de <i>polling</i> .
Solicitud equivalente reciente	<i>ALREADY_TRIGGERED</i>	No se inicia una nueva generación y se informa al usuario.
Ausencia de datos	<i>NO_DATA</i>	Se comunica que no existen datos para construir el informe.
Informe aún no disponible	<i>ready=false</i>	El cliente mantiene las consultas periódicas mientras no se alcance el tiempo máximo configurado.
Informe disponible	<i>ready=true</i>	Se obtiene el identificador y se inicia la descarga del fichero.
Tiempo máximo alcanzado	Timeout del <i>polling</i>	Se informa de que el informe puede consultarse posteriormente desde el historial.
Error al iniciar la generación	<i>ERROR</i>	Se presenta el mensaje de error y finaliza el seguimiento de la solicitud.

7.3 Página de Historial

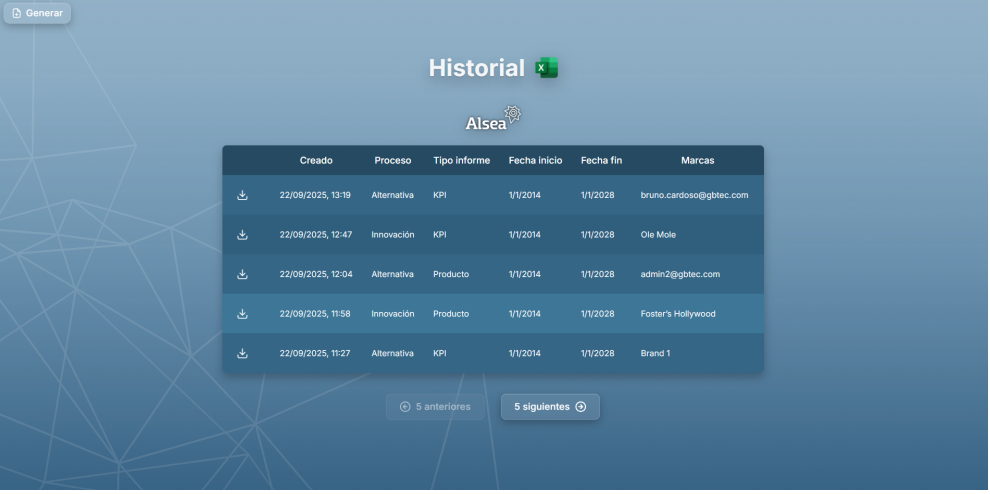
La página de historial es una funcionalidad incorporada durante este proyecto. Antes de la modernización no existía un mecanismo para consultar informes anteriores, por lo que el acceso posterior a la misma información requería una nueva generación.

7.3.1 Funcionalidad

- presenta los informes almacenados en forma de tabla;
- muestra fecha de creación, tipo, categoría, franquicias y fechas;
- ordena los registros de más reciente a más antiguo;
- incorpora controles de paginación;
- permite descargar directamente cualquier informe almacenado.

7.3.2 Comportamiento de la interfaz

La tabla concentra la información necesaria para identificar cada informe sin abrirlo. El botón de descarga utiliza el identificador persistente para recuperar directamente el contenido desde el backend, por lo que la operación no vuelve a ejecutar el proceso de generación.



Creado	Proceso	Tipo Informe	Fecha inicio	Fecha fin	Marcas
22/09/2025, 13:19	Alternativa	KPI	1/1/2014	1/1/2028	bruno.cardoso@gbtec.com
22/09/2025, 12:47	Innovación	KPI	1/1/2014	1/1/2028	Ole Mole
22/09/2025, 12:04	Alternativa	Producto	1/1/2014	1/1/2028	admin2@gbtec.com
22/09/2025, 11:58	Innovación	Producto	1/1/2014	1/1/2028	Foster's Hollywood
22/09/2025, 11:27	Alternativa	KPI	1/1/2014	1/1/2028	Brand 1

Figura 7.2: Vista de la página de historial de informes generados. Fuente: captura propia.

7.4 Flujo funcional final

La figura 7.3 resume el comportamiento global de la solución desde que el usuario inicia una generación hasta que el fichero puede descargarse o recuperarse posteriormente desde el historial.

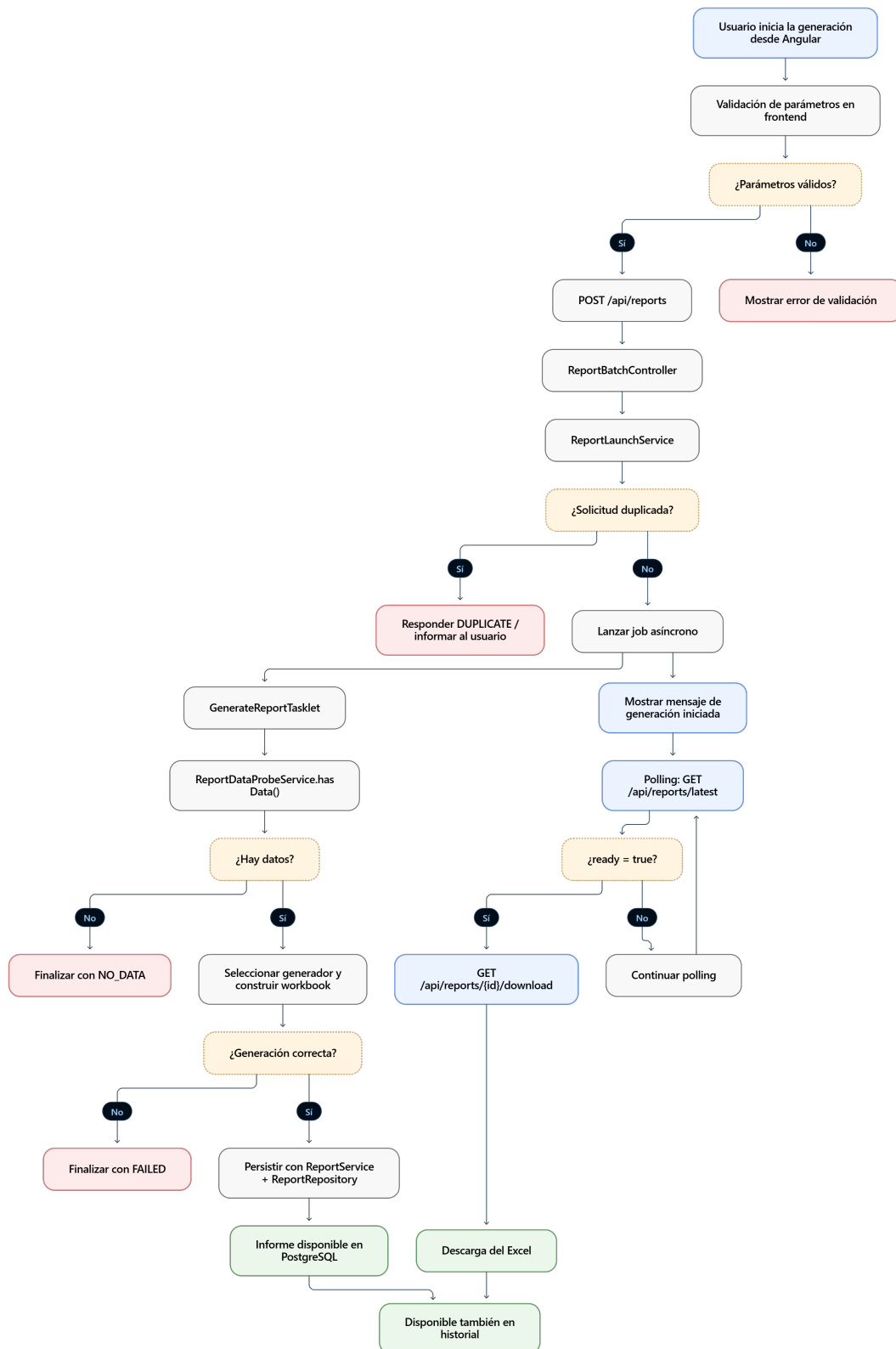


Figura 7.3: Flujo de interacción entre cliente y backend durante la generación, seguimiento y descarga de informes, generado con Mermaid. Fuente: elaboración propia.

El proceso comienza con la validación de los parámetros introducidos por el usuario. A continuación, el backend comprueba la posible duplicidad antes de lanzar el job asíncrono. Una vez iniciado, el frontend realiza el seguimiento mediante *polling* sobre */api/reports/latest*.

En paralelo, el backend comprueba la disponibilidad de datos, selecciona el generador adecuado y construye el workbook. Si la ejecución finaliza correctamente, el informe se persiste y pasa a estar disponible para su descarga. El flujo contempla igualmente los casos de parámetros inválidos, solicitud duplicada, ausencia de datos y error durante la generación.

El almacenamiento persistente permite que, incluso después de la descarga inicial o de finalizar el seguimiento del cliente, el documento pueda recuperarse posteriormente desde la página de historial.

7.5 Preparación de la release

Una vez completadas las funcionalidades y superadas las pruebas de QA, la última etapa consistió en preparar la nueva versión del Report Service según el proceso de release utilizado por el equipo.

El proceso seguía las convenciones de Gitflow y las políticas internas de versionado:

1. **Creación de la rama release candidate:** desde *development*, con los cambios previamente integrados y validados, se creaba una rama *release/vX.Y.Z-rc* destinada a los ajustes finales.
2. **Actualización de versión:** se modificaba el número de versión en los archivos de configuración correspondientes, incluyendo el *pom.xml* del backend.
3. **Builds y validaciones finales:** GitHub Actions compilaba el proyecto, ejecutaba las pruebas y comprobaba las métricas de calidad. También se realizaba una última validación funcional en QA.
4. **Integración en main:** se abría un Pull Request desde la rama de release y, tras la revisión, se integraba en la rama estable.
5. **Creación del tag:** el commit resultante se etiquetaba con el número de versión para identificar de manera precisa el código asociado a la publicación.
6. **Sincronización con development:** los ajustes propios de la release se integraban nuevamente para evitar divergencias.

7. **Documentación:** se actualizaban los README y la información de la versión con los cambios relevantes y las instrucciones necesarias.

Como parte del cierre se revisó y amplió la documentación de los repositorios frontend y backend. Los README incorporaron instrucciones de puesta en marcha, configuración, dependencias, comandos habituales, estructura general y consideraciones de despliegue, facilitando el mantenimiento posterior y la incorporación de nuevos desarrolladores.

Con este proceso, la versión desarrollada quedó preparada para su publicación, integrando persistencia histórica, procesamiento asíncrono, control de duplicidad, seguimiento desde el cliente e historial de informes.

Conclusión y trabajo futuro

8.1 Evaluación del cumplimiento de objetivos

Los objetivos definidos al inicio del trabajo pueden evaluarse a partir de las funcionalidades implementadas y de las evidencias obtenidas durante la validación. La tabla 8.1 resume el grado de cumplimiento de cada uno de ellos.

Tabla 8.1: Evaluación del cumplimiento de los objetivos del proyecto.

Objetivo inicial		Resultado	Evidencia principal
Incorporar	almacenamiento	Cumplido	PostgreSQL, Flyway, <i>ReportEntity</i> , <i>ReportRepository</i> , <i>ReportService</i> e historial.
Incrementar	mantenibilidad y	Cumplido	Cobertura aumentada del 17,7% al 25,7% y duplicación reducida del 15,8% al 10,0%.
Mejorar la	experiencia de	Cumplido	Evolución de la página de generación, mensajes de estado y nueva interfaz de historial.
Introducir	procesamiento	Cumplido	Integración con Spring Batch y seguimiento de la disponibilidad mediante <i>polling</i> .
Incorporar	control de duplici-	Cumplido	Implementación de <i>ReportDuplicateUtils</i> y consulta de metadatos mediante <i>JobExplorer</i> .
Permitir la	redescarga de in-	Cumplido	Persistencia de contenido binario y endpoint de descarga por UUID.

Los resultados muestran que los objetivos funcionales definidos para el proyecto fueron cubiertos dentro del periodo de desarrollo. No obstante, su cumplimiento debe interpretarse dentro de las limitaciones expuestas posteriormente, especialmente en relación con la ausencia de pruebas formales de carga, la cobertura parcial del código legado y las restricciones derivadas de la versión del frontend.

8.2 Conclusiones

Este Trabajo de Fin de Grado ha abordado la modernización del módulo *Report Service* integrado en la plataforma BIC para AESLA, dentro de un proyecto desarrollado durante tres meses de prácticas en GBTEC Software AG. Las tareas realizadas permitieron cubrir los principales objetivos planteados al inicio del proyecto y ampliar tanto las capacidades funcionales como la mantenibilidad del servicio.

Entre los principales resultados obtenidos destacan:

- **Persistencia e historial de informes:** se incorporó un almacenamiento persistente en PostgreSQL, gestionado mediante migraciones Flyway. Los informes generados pueden conservarse junto con sus metadatos y recuperarse posteriormente desde la nueva página de historial, evitando la necesidad de regenerarlos para volver a acceder a ellos.
- **Procesamiento asíncrono:** la generación se integró con Spring Batch, desacoplando el procesamiento del ciclo principal de la petición HTTP. El frontend realiza un seguimiento periódico mediante *polling*, permitiendo mantener la interfaz disponible mientras el informe se genera.
- **Control de duplicidad:** se incorporó una comprobación de solicitudes equivalentes dentro de una ventana temporal de cinco minutos, evitando iniciar una nueva generación cuando ya existe una solicitud reciente con los mismos parámetros.
- **Evolución de la interfaz:** se añadió una página específica de historial, nuevas validaciones y mensajes de estado asociados a la generación. Estas modificaciones proporcionan al usuario más información sobre el proceso y nuevas posibilidades de consulta y redescarga.
- **Mejora de las métricas de calidad:** la cobertura de código aumentó del 17.7% al 25.7%, superando el umbral mínimo del 20% definido por la empresa. Paralelamente, la duplicación detectada por SonarCloud se redujo del 15.8% al 10.0%.
- **Gestión de dependencias y vulnerabilidades:** la estandarización del gestor de paquetes en Yarn y el uso controlado de *resolutions* permitieron reducir los avisos de seguridad asociados a las dependencias, aunque permanecieron tres vulnerabilidades menores vinculadas al *tooling* de la versión utilizada de Angular.

Desde el punto de vista formativo, el proyecto permitió trabajar sobre una aplicación empresarial existente, donde las decisiones debían considerar no solo la implementación de nuevas funcionalidades, sino también la compatibilidad con código previo, la calidad, las

pruebas, la seguridad y el proceso de despliegue. El uso de Jira, Git, Pull Requests, revisión de código, pipelines de integración continua y entornos de QA proporcionó una visión práctica del ciclo de desarrollo de software en un entorno profesional.

En conjunto, la solución desarrollada amplía de forma significativa las capacidades del Report Service respecto a su estado inicial y establece una base más mantenible para futuras evoluciones. Los resultados obtenidos responden a los objetivos funcionales definidos para el proyecto, aunque existen limitaciones y líneas de mejora que se detallan a continuación.

8.3 Limitaciones

Aunque los objetivos principales del proyecto fueron alcanzados, el alcance temporal y el contexto de desarrollo condicionaron algunas de las decisiones adoptadas. Las principales limitaciones identificadas son las siguientes:

- **Ausencia de pruebas formales de carga:** se validó funcionalmente la generación asíncrona y la ejecución de los principales flujos, pero no se realizó un estudio específico de rendimiento bajo concurrencia elevada. Por tanto, no se dispone de métricas que permitan cuantificar el comportamiento del sistema ante un número elevado de generaciones simultáneas.
- **Versión del frontend:** el proyecto permanece sobre Angular 15. Aunque se mitigaron gran parte de los avisos de seguridad asociados a las dependencias, quedaron tres vulnerabilidades menores relacionadas con el *tooling* de esta versión.
- **Ausencia de una política de retención:** los informes almacenados permanecen actualmente en la base de datos sin un mecanismo automático de eliminación o una política de conservación configurable.
- **Seguimiento de la generación:** el estado se consulta mediante *polling* periódico. Aunque este mecanismo permite detectar cuándo el informe está disponible, no proporciona un porcentaje de progreso ni información detallada sobre las distintas etapas internas de generación.
- **Evaluación de usabilidad:** las interfaces fueron validadas funcionalmente dentro del proceso de QA, pero no se realizó un estudio formal de usabilidad con usuarios finales ni se recopilaban métricas cuantitativas de experiencia de usuario.

Estas limitaciones no impiden el funcionamiento de las funcionalidades desarrolladas, pero delimitan el alcance de las conclusiones que pueden extraerse y permiten identificar

posibles líneas de evolución del sistema.

8.4 Posibles ampliaciones futuras

Aunque el proyecto ha logrado cumplir con los objetivos planteados, existen diversas líneas de trabajo que podrían abordarse en el futuro para seguir mejorando el Report Service. Estas ampliaciones se han identificado durante el desarrollo del proyecto y representan oportunidades para aumentar la funcionalidad, seguridad y usabilidad del sistema.

8.4.1 Actualización de la versión de Angular

Como se indicó en el apartado de vulnerabilidades y en las limitaciones del proyecto, una de las principales mejoras pendientes consiste en migrar Angular 15 a una versión posterior compatible con el resto del stack y que disponga de soporte vigente.

Esta actualización permitiría:

- Reevaluar el árbol completo de dependencias del frontend.
- Reducir o eliminar, cuando sea posible, el bloque *resolutions* utilizado actualmente en *package.json*.
- Analizar nuevamente las vulnerabilidades residuales y utilizar versiones corregidas de las dependencias cuando estén disponibles.
- Beneficiarse de las mejoras introducidas en versiones posteriores del framework y de Angular Material.
- Reducir las incompatibilidades entre dependencias heredadas del proyecto.

La migración debería realizarse de forma progresiva, siguiendo las guías oficiales de actualización y validando el comportamiento de la aplicación tras cada cambio de versión. Debido a las posibles incompatibilidades entre dependencias, sería necesario acompañar la actualización de pruebas funcionales.

8.4.2 Mejoras en la gestión del historial

Una de las funcionalidades más solicitadas para futuras versiones es la ampliación de las capacidades de gestión del historial de informes. Específicamente, se ha identificado la necesidad de implementar:

Integración de la eliminación en el historial

El backend dispone actualmente de una operación `DELETE /api/reports/{id}` capaz de eliminar un informe persistido por su identificador. Sin embargo, esta capacidad no quedó integrada en la interfaz de historial dentro del alcance del proyecto.

Como ampliación futura se propone completar el flujo desde el frontend, incorporando una acción de eliminación para cada registro y un diálogo de confirmación previo a la operación. Esta confirmación permitiría reducir el riesgo de eliminaciones accidentales y proporcionar al usuario información sobre el resultado de la solicitud.

El diseño preliminar de esta funcionalidad incluiría:



	Creado	Proceso	Tipo informe	Fecha inicio	Fecha fin	Marcas
 	30/09/2025, 14:32	Alternativa	KPI	1/1/2014	1/1/2028	admin2@gbtec.com, admin@gbtec.com, Brand 1, bruno.cardoso@gbtec.com
 	30/09/2025, 14:32	Innovación	KPI	1/1/2014	1/1/2028	Domino's Pizza
 	30/09/2025, 10:55	Alternativa	Producto	1/1/2025	1/6/2025	Brand 1
 	30/09/2025, 10:54	Alternativa	KPI	1/1/2025	1/6/2025	admin2@gbtec.com
 	30/09/2025, 10:54	Innovación	KPI	1/1/2025	1/6/2025	Domino's Pizza

Figura 8.1: Mockup de la interfaz de historial con funcionalidad de eliminación de informes. Fuente: elaboración propia.

Como se observa en la figura 8.1, se añadiría una columna adicional con botones de eliminación para cada informe. Al pulsar el botón de borrar, aparecería un diálogo de confirmación para evitar eliminaciones accidentales.

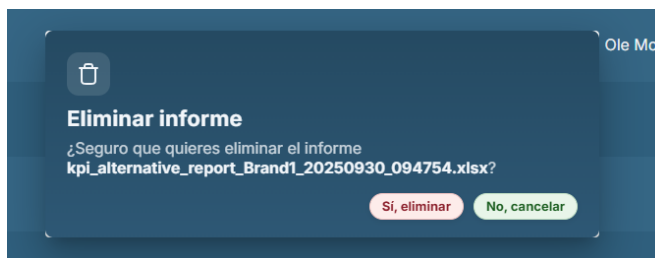


Figura 8.2: Diálogo de confirmación para la eliminación de informes. Fuente: elaboración propia.

La figura 8.2 muestra el diseño del diálogo de confirmación que solicitaría al usuario ve-

rificar la acción antes de eliminar permanentemente un informe del sistema.

Otras mejoras planificadas para el historial

Además de la funcionalidad de eliminación, se han identificado otras mejoras deseables:

- **Selección múltiple:** implementar checkboxes para seleccionar varios informes y realizar acciones en lote (eliminar múltiples informes, descargar varios informes simultáneamente).
- **Exposición de filtros avanzados en la interfaz:** el backend ya admite criterios de consulta por tipo, categoría, fecha de creación, nombre de fichero, fechas asociadas a la generación y franquicia. Como ampliación futura se propone incorporar estos criterios como controles visibles en la página de historial, permitiendo que el usuario construya búsquedas más precisas sin modificar el contrato REST existente.
- **Ordenación configurable desde la interfaz:** la consulta del historial utiliza *Pageable*, por lo que el backend está preparado para recibir información de paginación y ordenación. Una posible ampliación consistiría en permitir que el usuario seleccione directamente desde las cabeceras de la tabla el criterio y sentido de ordenación.
- **Exportación del listado:** posibilidad de exportar el listado del historial a CSV o Excel para auditorías o análisis externos.

8.4.3 Refactorización adicional del código backend

Aunque durante el proyecto se realizaron importantes tareas de refactorización que mejoraron las métricas de calidad del código, siempre existe margen de mejora. Las áreas identificadas para futuras mejoras incluyen:

- Mayor modularización de utilidades comunes.
- Ampliación de la cobertura de pruebas unitarias e integradas.
- Optimización de consultas a la base de datos.
- Incorporar una política de retención de datos.

8.4.4 Funcionalidades avanzadas

Mirando más allá de las mejoras inmediatas, se podrían considerar funcionalidades más ambiciosas:

- **Programación de informes:** permitir a los usuarios programar la generación automática de informes periódicos.
- **Notificaciones:** enviar notificaciones cuando un informe esté listo.
- **Plantillas personalizables:** permitir a los usuarios definir plantillas de informe con métricas específicas.
- **Dashboard analítico:** crear visualizaciones interactivas de las métricas de los informes sin necesidad de descargar Excel.
- **Control de acceso granular:** implementar permisos específicos por usuario o rol para la generación y consulta de informes.

Estas ampliaciones futuras representan oportunidades para seguir evolucionando el Report Service hacia una herramienta cada vez más completa y alineada con las necesidades cambiantes de AESLA.

Apéndices

Evidencias técnicas

A.1 Arquitectura detallada

La figura [A.1](#) amplía la representación simplificada presentada en el capítulo 5 e incorpora los principales componentes internos del frontend y del backend, el procesamiento asíncrono mediante Spring Batch, las fuentes de datos y los elementos de infraestructura utilizados durante el despliegue.

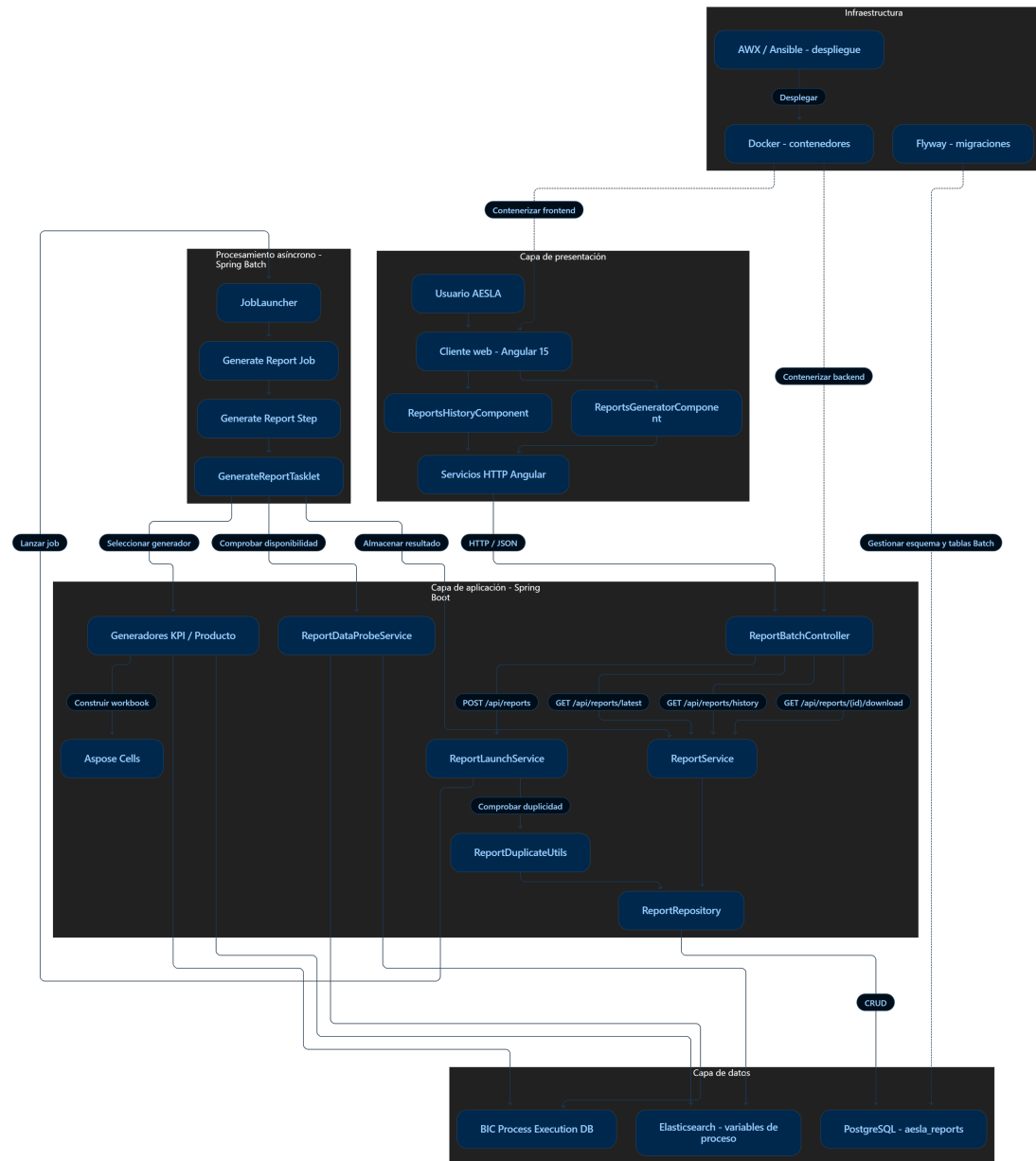


Figura A.1: Arquitectura detallada del Report Service. Fuente: elaboración propia.

En la capa de presentación, el usuario interactúa con los componentes Angular de generación e historial, que centralizan sus comunicaciones con el backend a través de servicios HTTP. El punto de entrada en la capa de aplicación es *ReportBatchController*, encargado de exponer las operaciones REST utilizadas por el cliente.

Para una nueva generación, el controlador delega en *ReportLaunchService*, que realiza la comprobación previa de duplicidad antes de iniciar el job de Spring Batch. El procesamiento

asíncrono se articula mediante *JobLauncher*, un job, un step y *GenerateReportTasklet*. Durante su ejecución se comprueba la disponibilidad de datos mediante *ReportDataProbeService* y se selecciona la lógica de generación correspondiente al tipo y categoría de informe.

Los datos utilizados proceden de BIC Process Execution y Elasticsearch, mientras que Aspose Cells proporciona las operaciones necesarias para la construcción de los libros Excel. Finalmente, *ReportService* centraliza el almacenamiento del resultado y utiliza *ReportRepository* para persistir el contenido y sus metadatos en el esquema *aesla_reports* de PostgreSQL.

La infraestructura complementaria incluye Flyway para gestionar las migraciones de base de datos, Docker para la contenerización de los servicios y AWX/Ansible para automatizar su despliegue en los diferentes entornos.

A.2 Evidencias de calidad de código

Las siguientes figuras recogen las capturas de SonarCloud utilizadas como evidencia de las métricas de cobertura y duplicación presentadas en el capítulo 6. En el cuerpo principal de la memoria se muestran únicamente los valores iniciales y finales, mientras que las capturas completas se trasladan a este apéndice para facilitar su consulta sin sobrecargar el capítulo de implementación y pruebas.



Figura A.2: Captura de SonarCloud correspondiente a la cobertura inicial del proyecto: 17.7%.



Figura A.3: Captura de SonarCloud correspondiente a la cobertura final del proyecto: 25.7%.



Figura A.4: Captura de SonarCloud correspondiente a la duplicación inicial del proyecto: 15.8%.



Figura A.5: Captura de SonarCloud correspondiente a la duplicación final del proyecto: 10.0%.

Lista de acrónimos

- API** Application Programming Interface. [13](#), [29](#)
- AWX** Interfaz web para la automatización con Ansible. [13](#)
- BPM** Business Process Management. [2](#)
- CI** Continuous Integration. [12](#), [13](#)
- CSS** Cascading Style Sheets. [13](#)
- EAM** Enterprise Architecture Management. [2](#)
- EPIC** Conjunto de historias de usuario agrupadas en Jira. [14](#), [15](#)
- GRC** Governance, Risk and Compliance. [2](#)
- HTML** HyperText Markup Language. [13](#)
- HTTP** HyperText Transfer Protocol. [29](#)
- JPA** Java Persistence API. [11](#)
- JSON** JavaScript Object Notation. [11](#), [29](#)
- KPI** Key Performance Indicator. [2](#)
- PR** Pull Request. [13](#), [15](#)
- QA** Quality Assurance. [13](#), [15](#)
- REST** Representational State Transfer. [29](#)
- SQL** Structured Query Language. [11](#)

Glosario

Angular Framework de desarrollo web basado en TypeScript para la construcción de aplicaciones cliente (*frontend*) modulares y escalables. [11](#)

Ansible Herramienta de automatización de configuración y despliegue de aplicaciones que utiliza archivos YAML para definir tareas y flujos de trabajo. [13](#)

Aspose Cells Biblioteca de Java empleada para la generación y manipulación avanzada de archivos en formato Excel. [11](#)

Code Smells Síntomas superficiales en el código fuente que pueden indicar problemas más profundos de diseño o implementación. Aunque no impiden el funcionamiento del software, los code smells dificultan la comprensión, el mantenimiento y la evolución del código. [13](#)

Coverage Métrica de calidad de software que mide el porcentaje de código fuente ejecutado durante las pruebas automatizadas. Un mayor coverage indica que una proporción más alta del código ha sido verificada mediante tests. [13](#)

development Rama principal de desarrollo utilizada para integrar los cambios aprobados antes de su incorporación a una versión de entrega. [42](#)

Docker Plataforma de contenedores que permite empaquetar aplicaciones y sus dependencias en un entorno aislado y reproducible. [15](#)

Duplicación de código Métrica que indica el porcentaje de código duplicado o repetido en un proyecto. La duplicación excesiva dificulta el mantenimiento, ya que los cambios deben replicarse en múltiples lugares, aumentando el riesgo de inconsistencias. [13](#)

Elasticsearch Motor de búsqueda y análisis distribuido basado en documentos JSON. Permite realizar búsquedas complejas y agregaciones de datos en tiempo real. [2](#)

- fix version** Versión de entrega utilizada en Jira para agrupar distintos work items que forman parte de una misma publicación o versión del software. [42](#)
- Gitflow** Convención y conjunto de prácticas para el uso de ramas en Git que organiza el desarrollo mediante ramas *feature*, *release*, *hotfix* y *develop/main*, facilitando la gestión de releases y mantenimiento. [12](#)
- GitHub Actions** Plataforma de integración continua y entrega continua (CI/CD) integrada en GitHub que permite definir flujos de trabajo (workflows) para compilar, probar y desplegar aplicaciones mediante acciones reutilizables. [12](#)
- Heatmap** Representación gráfica de datos en la que los valores se representan mediante colores, facilitando la identificación de patrones y tendencias. [2](#)
- Histograma** Representación gráfica de la distribución de datos mediante barras verticales u horizontales, donde cada barra representa la frecuencia o cantidad de elementos en un rango o categoría determinada. [5](#)
- JFrog Artifactory** Gestor universal de artefactos que almacena y gestiona paquetes, dependencias y artefactos binarios (imágenes Docker, paquetes Maven/NPM, etc.), facilitando el versionado y distribución en pipelines de CI/CD. [13](#)
- Jira** Herramienta de gestión ágil de proyectos que permite organizar tareas, historias de usuario y seguimiento de incidencias. [12](#), [14](#)
- Kanban** Metodología ágil de gestión visual del trabajo basada en tableros y tarjetas, que permite controlar el flujo de tareas en tiempo real. [12](#)
- PostgreSQL** Sistema de gestión de bases de datos relacional y de código abierto empleado para almacenar información estructurada. [12](#)
- Postman** Plataforma de colaboración para desarrollo de APIs que permite diseñar, probar, documentar y monitorizar servicios web mediante peticiones HTTP, colecciones de pruebas automatizadas y entornos configurables. [13](#)
- Quay** Registro de imágenes de contenedores (container registry) que permite almacenar, versionar y distribuir imágenes Docker en entornos empresariales. [13](#)
- Retroplanning** Técnica de planificación de proyectos que consiste en definir las tareas y hitos necesarios para alcanzar un objetivo, comenzando desde la fecha de finalización prevista y retrocediendo hasta el inicio del proyecto. [2](#)

SonarCloud Plataforma de análisis estático de código que ayuda a identificar problemas de calidad, vulnerabilidades y malas prácticas en el desarrollo de software. [13](#)

Spring Batch Framework de Spring orientado al procesamiento por lotes (*batch processing*), utilizado para ejecutar tareas asíncronas de larga duración. [6](#)

Bibliografía

- [1] Alsea, “Información corporativa y presencia internacional,” 2026, fuente corporativa de referencia; el nombre del cliente se ha anonimizado en este TFG. Accedido: 04-09-2026. [En línea]. Disponible en: <https://www.alsea.net/>
- [2] Object Management Group, “Business process model and notation (bpmn),” 2014, accedido: 04-09-2026. [En línea]. Disponible en: <https://www.omg.org/spec/BPMN/2.0.2/>
- [3] GBTEC Software AG, “Gbtec software,” 2026, accedido: 04-09-2026. [En línea]. Disponible en: <https://www.gbtec.com/>
- [4] —, “Gbtec platform,” 2026, anteriormente denominada BIC Platform. Accedido: 06-09-2026. [En línea]. Disponible en: <https://www.gbtec.com/software/>
- [5] Microsoft, “Power bi documentation,” 2026, accedido: 04-09-2026. [En línea]. Disponible en: <https://learn.microsoft.com/en-us/power-bi/>
- [6] Salesforce, “Tableau documentation,” 2026, accedido: 04-09-2026. [En línea]. Disponible en: <https://www.tableau.com/support>
- [7] Qlik, “Qlik sense documentation,” 2026, accedido: 04-09-2026. [En línea]. Disponible en: <https://help.qlik.com/en-US/cloud-services/>
- [8] Jaspersoft, “Jasperreports library,” 2026, repositorio y documentación oficial. Accedido: 06-09-2026. [En línea]. Disponible en: <https://github.com/Jaspersoft/jasperreports>
- [9] Apache Software Foundation, “Apache poi - java api for microsoft documents,” 2026, documentación oficial de Apache POI. Accedido: 05-09-2026. [En línea]. Disponible en: <https://poi.apache.org/>
- [10] Spring Team, “Spring boot reference documentation,” 2026, accedido: 05-09-2026. [En línea]. Disponible en: <https://docs.spring.io/spring-boot/documentation.html>

- [11] —, “Spring batch reference documentation,” 2026, accedido: 04-09-2026. [En línea]. Disponible en: <https://docs.spring.io/spring-batch/reference/>
- [12] Hibernate Team, “Hibernate orm documentation,” 2026, accedido: 04-09-2026. [En línea]. Disponible en: <https://hibernate.org/orm/documentation/>
- [13] Aspose, “Aspose.cells for java documentation,” 2026, accedido: 04-09-2026. [En línea]. Disponible en: <https://docs.aspose.com/cells/java/>
- [14] Elastic, “Elasticsearch reference,” 2026, accedido: 04-09-2026. [En línea]. Disponible en: <https://www.elastic.co/guide/en/elasticsearch/reference/current/index.html>
- [15] Apache Maven Project, “Maven documentation,” 2026, accedido: 04-09-2026. [En línea]. Disponible en: <https://maven.apache.org/guides/>
- [16] Google, “Angular documentation,” 2026, accedido: 04-09-2026. [En línea]. Disponible en: <https://angular.dev/overview>
- [17] PostgreSQL Global Development Group, “Postgresql documentation,” 2026, accedido: 04-09-2026. [En línea]. Disponible en: <https://www.postgresql.org/docs/>
- [18] Redgate, “Flyway documentation,” 2026, accedido: 05-09-2026. [En línea]. Disponible en: <https://documentation.red-gate.com/flyway>
- [19] Testcontainers, “Testcontainers documentation,” 2026, accedido: 04-09-2026. [En línea]. Disponible en: <https://testcontainers.com/guides/>
- [20] Docker, “Docker documentation,” 2026, accedido: 04-09-2026. [En línea]. Disponible en: <https://docs.docker.com/>
- [21] Atlassian, “Jira software documentation,” 2026, accedido: 04-09-2026. [En línea]. Disponible en: <https://support.atlassian.com/jira-software-cloud/>
- [22] V. Driessen, “A successful git branching model,” *nvie.com*, 2010, accedido: 15-11-2025. [En línea]. Disponible en: <https://nvie.com/posts/a-successful-git-branching-model/>
- [23] GitHub, “Github actions documentation,” 2026, accedido: 04-09-2026. [En línea]. Disponible en: <https://docs.github.com/en/actions>
- [24] SonarSource, “Sonarqube documentation,” 2026, accedido: 04-09-2026. [En línea]. Disponible en: <https://docs.sonarsource.com/sonarqube/>
- [25] K. Beck, M. Beedle, A. van Bennekum, A. Cockburn, W. Cunningham, M. Fowler, J. Grenning, J. Highsmith, A. Hunt, R. Jeffries, J. Kern, B. Marick, R. C. Martin, S. Mellor,

- K. Schwaber, J. Sutherland, and D. Thomas, “Manifesto for agile software development,” 2001, accedido: 15-11-2025. [En línea]. Disponible en: <https://agilemanifesto.org/iso/es/manifesto.html>
- [26] D. J. Anderson, *Kanban: Successful Evolutionary Change for Your Technology Business*. Blue Hole Press, 2010.
- [27] Google, “Angular material documentation,” 2026, accedido: 04-09-2026. [En línea]. Disponible en: <https://material.angular.dev/>
- [28] ReactiveX, “Rxjs documentation,” 2026, accedido: 04-09-2026. [En línea]. Disponible en: <https://rxjs.dev/>
- [29] Redgate, “Versioned migrations,” 2024, accedido: 05-09-2026. [En línea]. Disponible en: <https://documentation.red-gate.com/fd/versioned-migrations-273973333.html>
- [30] Spring Team, “Meta-data schema :: Spring batch reference,” 2026, accedido: 05-09-2026. [En línea]. Disponible en: <https://docs.spring.io/spring-batch/reference/schema-appendix.html>